HYPERCTL-STAR SATISFIABILITY IS SIGMA-1-2-COMPLETE

MARIE FORTIN, LOUWE B. KUIJER, PATRICK TOTZKE, MARTIN ZIMMERMANN

Allowed standard prerequisites (editorial).

- Arithmetic hierarchies, temporal logic and standard coding of arithmetic. Original bibliography retained; the cited HyperLTL arithmetic construction is a named prerequisite; all author-local definitions, reductions, witness-tree arguments and continuations are retained.

1. Introduction

Most classical temporal logics like LTL and CTL* refer to a single execution trace at a time while information-flow properties, which are crucial for security-critical systems, require reasoning about multiple executions of a system. Clarkson and Schneider [CS10] coined the term *hyperproperties* for such properties which, structurally, are sets *of sets* of traces. Just like ordinary trace and branching-time properties, hyperproperties can be specified using temporal logics, e.g. HyperLTL and HyperCTL* [CFK$^+$14], expressive, but intuitive specification languages that are able to express typical information-flow properties such as noninterference, non-inference, declassification, and input determinism. Due to their practical relevance and theoretical elegance, hyperproperties and their specification languages have received considerable attention during the last decade.

HyperLTL is obtained by extending LTL [Pnu77], the most influential specification language for linear-time properties, by trace quantifiers to refer to multiple executions of a system. For example, the HyperLTL formula

$$\forall \pi, \pi'.\ \mathbf{G}(i_\pi \leftrightarrow i_{\pi'}) \rightarrow \mathbf{G}(o_\pi \leftrightarrow o_{\pi'})$$

expresses input determinism, i.e. every pair of traces that always has the same input (represented by the proposition i) also always has the same output (represented by the proposition o). Similarly, HyperCTL* is the extension of the branching-time logic CTL* [EH86] by path quantifiers. HyperLTL only allows formulas in prenex normal form while HyperCTL* allows arbitrary quantification, in particular under the scope of temporal operators. Consequently, HyperLTL formulas are evaluated over sets of traces while HyperCTL* formulas are evaluated over transition systems, which yield the underlying branching structure of the traces.

All basic verification problems, e.g. model checking [BF22, BF23a, Fin21, FRS15], runtime monitoring [AB16, BF16, BSB17, BF18, CFH$^+$21, FHST18], and synthesis [BF20, FHHT20, FHL$^+$20], have been studied. Most importantly, HyperCTL* model checking over finite transition systems is TOWER-complete [FRS15], even for a fixed transition system [MZ20]. However, for a small number of alternations, efficient algorithms have been developed and were applied to a wide range of problems, e.g. an information-flow analysis of an I2C bus master [FRS15], the symmetric access to a shared resource in a mutual exclusion protocol [FRS15], and to detect the use of a defeat device to cheat in emission testing [BDFH16].

But surprisingly, the exact complexity of the satisfiability problems for HyperLTL and HyperCTL* is still open. Finkbeiner and Hahn proved that HyperLTL

Date: arxiv:2303.16699v5; CC BY 4.0.

satisfiability is undecidable [FH16], a result which already holds when only considering finite sets of ultimately periodic traces and $\forall\exists$-formulas. In fact, Finkbeiner et al. showed that HyperLTL satisfiability restricted to finite sets of ultimately periodic traces is Σ^0_1-complete [FHH18] (i.e. complete for the set of recursively enumerable problems). Furthermore, Hahn and Finkbeiner proved that the $\exists^*\forall^*$-fragment has decidable satisfiability [FH16] while Mascle and Zimmermann studied the HyperLTL satisfiability problem restricted to bounded sets of traces [MZ20]. The latter work implies that HyperLTL satisfiability restricted to finite sets of traces (even non ultimately periodic ones) is also Σ^0_1-complete. Following up on the results presented in the conference version of this article [FKTZ21], Beutner et al. studied satisfiability for safety and liveness fragments of HyperLTL [BCF$^+$22]. Finally, Finkbeiner et al. developed tools and heuristics [BF23a, FHH18, FHS17].

As every HyperLTL formula can be turned into an equisatisfiable HyperCTL* formula, HyperCTL* satisfiability is also undecidable. Moreover, Rabe has shown that it is even Σ^1_1-hard [Rab16], i.e. it is not even arithmetical. However, both for HyperLTL and for HyperCTL* satisfiability, only lower bounds, but no upper bounds, are known.

Our Contributions. In this paper, we settle the complexity of the satisfiability problems for HyperLTL and HyperCTL* by determining exactly how undecidable they are. That is, we provide matching lower and upper bounds in terms of the analytical hierarchy and beyond, where decision problems (encoded as subsets of $\mathbb{N}$) are classified based on their definability by formulas of higher-order arithmetic, namely by the type of objects one can quantify over and by the number of alternations of such quantifiers. We refer to Roger's textbook [Rog87] for fully formal definitions. For our purposes, it suffices to recall the following classes. Σ^0_1 contains the sets of natural numbers of the form

$$\{x \in \mathbb{N} \mid \exists x_0.\ \cdots \exists x_k.\ \psi(x, x_0, \ldots, x_k)\}$$

where quantifiers range over natural numbers and ψ is a quantifier-free arithmetic formula. The notation Σ^0_1 signifies that there is a single block of existential quantifiers (the subscript 1) ranging over natural numbers (type 0 objects, explaining the superscript 0). Analogously, Σ^1_1 is induced by arithmetic formulas with existential quantification of type 1 objects (functions mapping natural numbers to natural numbers) and arbitrary (universal and existential) quantification of type 0 objects. Finally, Σ^2_1 is induced by arithmetic formulas with existential quantification of type 2 objects (functions mapping type 1 objects to natural numbers) and arbitrary quantification of type 0 and type 1 objects. So, Σ^0_1 is part of the first level of the arithmetic hierarchy, Σ^1_1 is part of the first level of the analytical hierarchy, while Σ^2_1 is not even analytical.

In terms of this classification, we prove that HyperLTL satisfiability is Σ^1_1-complete while HyperCTL* satisfiability is Σ^2_1-complete, thereby settling the complexity of both problems and showing that they are highly undecidable. In both cases, this is a significant increase of the lower bound and the first upper bound.

First, let us consider HyperLTL satisfiability. The Σ^1_1 lower bound is a straightforward reduction from the recurrent tiling problem, a standard Σ^1_1-complete problem asking whether $\mathbb{N} \times \mathbb{N}$ can be tiled by a given finite set of tiles. So, let us consider the upper bound: Σ^1_1 allows to quantify over type 1 objects: functions from natural numbers to natural numbers, or, equivalently, over sets of natural numbers, i.e. countable objects. On the other hand, HyperLTL formulas are evaluated over sets of infinite traces, i.e. uncountable objects. Thus, to show that quantification over type 1 objects is sufficient, we need to apply a result of Finkbeiner and Zimmermann proving that every satisfiable HyperLTL formula has a countable model [FZ17]. Then, we can prove Σ^1_1-membership by expressing the existence of

a model and the existence of appropriate Skolem functions for the trace quantifiers by type 1 quantification. We also prove that the satisfiability problem remains Σ^1_1-complete when restricted to ultimately periodic traces, or, equivalently, when restricted to finite traces.

Then, we turn our attention to HyperCTL* satisfiability. Recall that HyperCTL* formulas are evaluated over (possibly infinite) transition systems, which can be much larger than type 2 objects, as the cardinality of type 2 objects is bounded by $\mathfrak{c}$, the cardinality of the continuum. Hence, to obtain our upper bound on the complexity we need, just like in the case of HyperLTL, an upper bound on the size of minimal models of satisfiable HyperCTL* formulas. To this end, we generalise the proof of Finkbeiner and Zimmermann to HyperCTL*, showing that every satisfiable HyperCTL* formula has a model of size $\mathfrak{c}$. We also exhibit a satisfiable HyperCTL* formula $\varphi_{\mathfrak{c}}$ whose models all have at least cardinality $\mathfrak{c}$, as they have to encode all subsets of $\mathbb{N}$ by disjoint paths. Thus, our upper bound $\mathfrak{c}$ is tight.

With this upper bound on the cardinality of models, we are able to prove Σ^2_1-membership of HyperCTL* satisfiability by expressing with type 2 quantification the existence of a model and the existence of a winning strategy in the induced model checking game. The matching lower bound is proven by directly encoding the arithmetic formulas inducing Σ^2_1 as instances of the HyperCTL* satisfiability problem. To this end, we use the formula $\varphi_{\mathfrak{c}}$ whose models have for each subset $A \subseteq \mathbb{N}$ a path encoding A. Now, quantification over type 0 objects (natural numbers) is simulated by quantification of a path encoding a singleton set, quantification over type 1 objects (which can be assumed to be sets of natural numbers) is simulated by quantification over the paths encoding such subsets, and existential quantification over type 2 objects (which can be assumed to be subsets of $2^{\mathbb{N}}$) is simulated by the choice of the model, i.e. a model encodes k subsets of $2^{\mathbb{N}}$ if there are k existential type 2 quantifiers. Finally, the arithmetic operations can easily be implemented in HyperLTL, and therefore also in HyperCTL*.

Using variations of these techniques, we also show that HyperCTL* satisfiability restricted to countable or to finitely branching models is equivalent to truth in second-order arithmetic, i.e. the question whether a given sentence of second-order arithmetic is satisfied in the structure $(\mathbb{N}, 0, 1, +, \cdot, <)$. Restricting the class of models makes the problem simpler, but it is still highly-undecidable.

After settling the complexity of satisfiability, we turn our attention to the HyperLTL quantifier alternation hierarchy and its relation to satisfiability. Rabe remarks that the hierarchy is strict [Rab16]. On the other hand, Mascle and Zimmermann show that every HyperLTL formula has a polynomial-time computable equi-satisfiable formula with one quantifier alternation [MZ20]. Here, we present a novel proof of strictness by embedding the FO[$<$] alternation hierarchy, which is also strict [CB71, Tho81]. We use our construction to prove that for every $n > 0$, deciding whether a given formula is equivalent to a formula with at most n quantifier alternations is Π^1_1-complete (Π^1_1 is the co-class of Σ^1_1, i.e. containing the complements of sets in Σ^1_1).

2. Preliminaries

Fix a finite set AP of atomic propositions. A *trace* over AP is a map $t\colon \mathbb{N} \to 2^{\mathrm{AP}}$, denoted by $t(0)t(1)t(2)\cdots$. It is *ultimately periodic*, if $t = x \cdot y^\omega$ for some $x, y \in (2^{\mathrm{AP}})^+$, i.e. there are $s, p > 0$ with $t(n) = t(n+p)$ for all $n \geq s$. The set of all traces over AP is $(2^{\mathrm{AP}})^\omega$.

A transition system $\mathcal{T} = (V, E, v_I, \lambda)$ consists of a non-empty set V of vertices, a set $E \subseteq V \times V$ of (directed) edges, an initial vertex $v_I \in V$, and a labelling $\lambda\colon V \to 2^{\mathrm{AP}}$ of the vertices by sets of atomic propositions. We require that each vertex has at

least one outgoing edge. A path ρ through $\mathcal{T}$ is an infinite sequence $\rho(0)\rho(1)\rho(2)\cdots$ of vertices with $(\rho(n), \rho(n+1)) \in E$ for every $n \geq 0$. The trace of ρ is defined as $\lambda(\rho(0))\lambda(\rho(1))\lambda(\rho(2))\cdots$.

2.1. **HyperLTL.** The formulas of HyperLTL are given by the grammar

$$\varphi ::= \exists \pi.\ \varphi \mid \forall \pi.\ \varphi \mid \psi \qquad\qquad \psi ::= a_\pi \mid \neg\psi \mid \psi \vee \psi \mid \mathbf{X}\,\psi \mid \psi\,\mathbf{U}\,\psi$$

where a ranges over atomic propositions in AP and where π ranges over a fixed countable set $\mathcal{V}$ of *(trace) variables*. Conjunction, implication, and equivalence are defined as usual, and the temporal operators eventually $\mathbf{F}$ and always $\mathbf{G}$ are derived as $\mathbf{F}\,\psi = \neg\psi\,\mathbf{U}\,\psi$ and $\mathbf{G}\,\psi = \neg\,\mathbf{F}\,\neg\psi$. A *sentence* is a formula without free variables.

The semantics of HyperLTL is defined with respect to a *trace assignment*, a partial mapping $\Pi\colon \mathcal{V} \to (2^{\mathrm{AP}})^\omega$. The assignment with empty domain is denoted by $\Pi_\emptyset$. Given a trace assignment Π, a variable π, and a trace t we denote by $\Pi[\pi \to t]$ the assignment that coincides with Π everywhere but at π, which is mapped to t. Furthermore, $\Pi[j, \infty)$ denotes the trace assignment mapping every π in Π's domain to $\Pi(\pi)(j)\Pi(\pi)(j+1)\Pi(\pi)(j+2)\cdots$, its suffix from position j onwards.

For sets T of traces and trace assignments Π we define

- $(T, \Pi) \models a_\pi$ if $a \in \Pi(\pi)(0)$,
- $(T, \Pi) \models \neg\psi$ if $(T, \Pi) \not\models \psi$,
- $(T, \Pi) \models \psi_1 \vee \psi_2$ if $(T, \Pi) \models \psi_1$ or $(T, \Pi) \models \psi_2$,
- $(T, \Pi) \models \mathbf{X}\,\psi$ if $(T, \Pi[1, \infty)) \models \psi$,
- $(T, \Pi) \models \psi_1\,\mathbf{U}\,\psi_2$ if there is a $j \geq 0$ such that $(T, \Pi[j, \infty)) \models \psi_2$ and for all $0 \leq j' < j$: $(T, \Pi[j', \infty)) \models \psi_1$,
- $(T, \Pi) \models \exists \pi.\ \varphi$ if there exists a trace $t \in T$ such that $(T, \Pi[\pi \to t]) \models \varphi$, and
- $(T, \Pi) \models \forall \pi.\ \varphi$ if for all traces $t \in T$: $(T, \Pi[\pi \to t]) \models \varphi$.

We say that T *satisfies* a sentence φ if $(T, \Pi_\emptyset) \models \varphi$. In this case, we write $T \models \varphi$ and say that T is a *model* of φ. Two HyperLTL sentences φ and φ' are equivalent if $T \models \varphi$ if and only if $T \models \varphi'$ for every set T of traces. Although HyperLTL sentences are required to be in prenex normal form, they are closed under Boolean combinations, which can easily be seen by transforming such a formula into an equivalent formula in prenex normal form.

2.2. **HyperCTL*.** The formulas of HyperCTL* are given by the grammar

$$\varphi ::= a_\pi \mid \neg\varphi \mid \varphi \vee \varphi \mid \mathbf{X}\,\varphi \mid \varphi\,\mathbf{U}\,\varphi \mid \exists \pi.\ \varphi \mid \forall \pi.\ \varphi$$

where a ranges over atomic propositions in AP and where π ranges over a fixed countable set $\mathcal{V}$ of *(path) variables*, and where we require that each temporal operator appears in the scope of a path quantifier. Again, other Boolean connectives and temporal operators are derived as usual. Sentences are formulas without free variables.

Let $\mathcal{T}$ be a transition system. The semantics of HyperCTL* is defined with respect to a *path assignment*, a partial mapping Π from variables in $\mathcal{V}$ to paths of $\mathcal{T}$. The assignment with empty domain is denoted by $\Pi_\emptyset$. Given a path assignment Π, a variable π, and a path ρ we denote by $\Pi[\pi \to \rho]$ the assignment that coincides with Π everywhere but at π, which is mapped to ρ. Furthermore, $\Pi[j, \infty)$ denotes the path assignment mapping every π in Π's domain to $\Pi(\pi)(j)\Pi(\pi)(j+1)\Pi(\pi)(j+2)\cdots$, its suffix from position j onwards.

For transition systems $\mathcal{T}$ and path assignments Π we define

- $(\mathcal{T}, \Pi) \models a_\pi$ if $a \in \lambda(\Pi(\pi)(0))$, where λ is the labelling function of $\mathcal{T}$,
- $(\mathcal{T}, \Pi) \models \neg\psi$ if $(\mathcal{T}, \Pi) \not\models \psi$,

- $(\mathcal{T}, \Pi) \models \psi_1 \vee \psi_2$ if $(\mathcal{T}, \Pi) \models \psi_1$ or $(\mathcal{T}, \Pi) \models \psi_2$,
- $(\mathcal{T}, \Pi) \models \mathbf{X}\,\psi$ if $(\mathcal{T}, \Pi[1, \infty)) \models \psi$,
- $(\mathcal{T}, \Pi) \models \psi_1\,\mathbf{U}\,\psi_2$ if there exists a $j \geq 0$ such that $(\mathcal{T}, \Pi[j, \infty)) \models \psi_2$ and for all $0 \leq j' < j$: $(\mathcal{T}, \Pi[j', \infty)) \models \psi_1$,
- $(\mathcal{T}, \Pi) \models \exists \pi.\ \varphi$ if there exists a path ρ of $\mathcal{T}$, starting in $\mathrm{rcnt}(\Pi)$, such that $(\mathcal{T}, \Pi[\pi \to \rho]) \models \varphi$, and
- $(\mathcal{T}, \Pi) \models \forall \pi.\ \varphi$ if for all paths ρ of $\mathcal{T}$ starting in $\mathrm{rcnt}(\Pi)$: $(\mathcal{T}, \Pi[\pi \to \rho]) \models \varphi$.

Here, $\mathrm{rcnt}(\Pi)$ is the initial vertex of $\Pi(\pi)$, where π is the path variable most recently added to or changed in Π, and the initial vertex of $\mathcal{T}$ if Π is empty.[1] We say that $\mathcal{T}$ *satisfies* a sentence φ if $(\mathcal{T}, \Pi_\emptyset) \models \varphi$. In this case, we write $\mathcal{T} \models \varphi$ and say that $\mathcal{T}$ is a *model* of φ.

2.3. **Complexity Classes for Undecidable Problems.** A type 0 object is a natural number $n \in \mathbb{N}$, a type 1 object is a function $f\colon \mathbb{N} \to \mathbb{N}$, and a type 2 object is a function $f\colon (\mathbb{N} \to \mathbb{N}) \to \mathbb{N}$. As usual, predicate logic with quantification over type 0 objects (first-order quantifiers) is called first-order logic. Second- and third-order logic are defined similarly.

We consider formulas of arithmetic, i.e. predicate logic with signature $(0, 1, +, \cdot, <)$ evaluated over the natural numbers. With a single free variable of type 0, such formulas define sets of natural numbers (see, e.g. Rogers [Rog87] for more details):

- Σ^0_1 contains the sets of the form $\{x \in \mathbb{N} \mid \exists x_0.\ \cdots \exists x_k.\ \psi(x, x_0, \ldots, x_k)\}$ where ψ is a quantifier-free arithmetic formula and the x_i are variables of type 0.
- Σ^1_1 contains the sets of the form $\{x \in \mathbb{N} \mid \exists x_0.\ \cdots \exists x_k.\ \psi(x, x_0, \ldots, x_k)\}$ where ψ is an arithmetic formula with arbitrary (existential and universal) quantification over type 0 objects and the x_i are variables of type 1.
- Σ^2_1 contains the sets of the form $\{x \in \mathbb{N} \mid \exists x_0.\ \cdots \exists x_k.\ \psi(x, x_0, \ldots, x_k)\}$ where ψ is an arithmetic formula with arbitrary (existential and universal) quantification over type 0 and type 1 objects and the x_i are variables of type 2.

Note that there is a bijection between functions of the form $f\colon \mathbb{N} \to \mathbb{N}$ and subsets of $\mathbb{N}$, which is implementable in arithmetic. Similarly, there is a bijection between functions of the form $f\colon (\mathbb{N} \to \mathbb{N}) \to \mathbb{N}$ and subsets of $2^{\mathbb{N}}$, which is again implementable in arithmetic. Thus, whenever convenient, we use quantification over sets of natural numbers and over sets of sets of natural numbers, instead of quantification over type 1 and type 2 objects; in particular when proving lower bounds. We then include $\in$ in the signature.

Also, note that 0 and 1 are definable in first-order arithmetic. Thus, whenever convenient, we drop 0 and 1 from the signature of arithmetic. In the same vein, every fixed natural number is definable in first-order arithmetic.

3. HyperLTL satisfiability is Σ^1_1-complete

In this section, we settle the complexity of the satisfiability problem for HyperLTL: given a HyperLTL sentence, determine whether it has a model.

Theorem 3.1. *HyperLTL satisfiability is Σ^1_1-complete.*

We should contrast this result with the Σ^0_1-completeness of HyperLTL satisfiability restricted to *finite* sets of ultimately periodic traces [FHH18, Theorem 1]. The

[1] For the sake of simplicity, we refrain from formalising this notion properly, which would require to keep track of the order in which variables are added to or changed in Π.

Σ^1_1-completeness of HyperLTL satisfiability in the general case implies that, in particular, the set of satisfiable HyperLTL sentences is neither recursively enumerable nor co-recursively enumerable. A semi-decision procedure, like the one introduced in [FHH18] for finite sets of ultimately periodic traces, therefore cannot exist in general.

3.1. **HyperLTL satisfiability is in Σ^1_1.** The Σ^1_1 upper bound relies on the fact that every satisfiable HyperLTL formula has a countable model [FZ17]. This allows us to represent these models, and Skolem functions on them, by sets of natural numbers, which are type 1 objects. In this encoding, trace assignments are type 0 objects, as traces in a countable set can be identified by natural numbers. With some more existential type 1 quantification one can then express the existence of a function witnessing that every trace assignment consistent with the Skolem functions satisfies the quantifier-free part of the formula under consideration.

Lemma 3.2. *HyperLTL satisfiability is in Σ^1_1.*

Proof. Let φ be a HyperLTL formula, let Φ denote the set of quantifier-free subformulas of φ, and let Π be a trace assignment whose domain contains the variables of φ. The expansion of φ on Π is the function $e_{\varphi,\Pi}\colon \Phi \times \mathbb{N} \to \{0,1\}$ with

$$e_{\varphi,\Pi}(\psi, j) = \begin{cases} 1 & \text{if } \Pi[j,\infty) \models \psi, \text{ and} \\ 0 & \text{otherwise.} \end{cases}$$

The expansion is completely characterised by the following consistency conditions:

- $e_{\varphi,\Pi}(a_\pi, j) = 1$ if and only if $a \in \Pi(\pi)(j)$.
- $e_{\varphi,\Pi}(\neg\psi, j) = 1$ if and only if $e_{\varphi,\Pi}(\psi, j) = 0$.
- $e_{\varphi,\Pi}(\psi_1 \vee \psi_2, j) = 1$ if and only if $e_{\varphi,\Pi}(\psi_1, j) = 1$ or $e_{\varphi,\Pi}(\psi_2, j) = 1$.
- $e_{\varphi,\Pi}(\mathbf{X}\,\psi, j) = 1$ if and only if $e_{\varphi,\Pi}(\psi, j+1) = 1$.
- $e_{\varphi,\Pi}(\psi_1 \,\mathbf{U}\, \psi_2, j) = 1$ if and only if there is a $j' \geq j$ such that $e_{\varphi,\Pi}(\psi_2, j') = 1$ and $e_{\varphi,\Pi}(\psi_2, j'') = 1$ for all j'' in the range $j \leq j'' < j'$.

Every satisfiable HyperLTL sentence has a countable model [FZ17]. Hence, to prove that the HyperLTL satisfiability problem is in Σ^1_1, we express, for a given HyperLTL sentence φ encoded as a natural number, the existence of the following type 1 objects (relying on the fact that there is a bijection between finite sequences over $\mathbb{N}$ (denoted by $\mathbb{N}^*$) and $\mathbb{N}$ itself):

- A countable set of traces over the propositions of φ encoded as a function T from $\mathbb{N} \times \mathbb{N}$ to $\mathbb{N}$, mapping trace names and positions to (encodings of) subsets of the set of propositions appearing in φ.
- A function S from $\mathbb{N} \times \mathbb{N}^*$ to $\mathbb{N}$ to be interpreted as Skolem functions for the existentially quantified variables of φ, i.e. we map a variable (identified by a natural number) and a trace assignment of the variables preceding it (encoded as a sequence of natural numbers) to a trace name.
- A function E from $\mathbb{N} \times \mathbb{N} \times \mathbb{N}$ to $\mathbb{N}$, where, for a fixed $a \in \mathbb{N}$ encoding a trace assignment Π, the function $x, y \mapsto E(a, x, y)$ is interpreted as the expansion of φ on Π, i.e. x encodes a subformula in Φ and y is a position.

Then, we express the following properties using only type 0 quantification: For every trace assignment of the variables in φ, encoded by $a \in \mathbb{N}$, if a is consistent with the Skolem function encoded by S, then the function $x, y \mapsto E(a, x, y)$ satisfies the consistency conditions characterising the expansion, and we have $E(a, x_0, 0) = 1$, where x_0 is the encoding of the maximal quantifier-free subformula of φ. We leave the tedious, but standard, details to the industrious reader. □

3.2. **HyperLTL satisfiability is Σ^1_1-hard.** To prove a matching lower bound, we reduce from the recurrent tiling problem [Har85], a standard Σ^1_1-complete problem.

Lemma 3.3. *HyperLTL satisfiability is Σ^1_1-hard.*

Proof. A *tile* is a function $\tau\colon \{east, west, north, south\} \to \mathcal{C}$ that maps directions into a finite set $\mathcal{C}$ of colours. Given a finite set Ti of tiles, a *tiling of the positive quadrant* with Ti is a function $ti\colon \mathbb{N}\times\mathbb{N} \to Ti$ with the property that:

- if $ti(i,j) = \tau_1$ and $ti(i+1,j) = \tau_2$, then $\tau_1(east) = \tau_2(west)$ and
- if $ti(i,j) = \tau_1$ and $ti(i,j+1) = \tau_2$, then $\tau_1(north) = \tau_2(south)$.

The *recurring tiling problem* is to determine, given a finite set Ti of tiles and a designated $\tau_0 \in Ti$, whether there is a tiling ti of the positive quadrant with Ti such that there are infinitely many $j \in \mathbb{N}$ such that $ti(0,j) = \tau_0$. This problem is known to be Σ^1_1-complete [Har85], so reducing it to HyperLTL satisfiability will establish the desired hardness result.

In our reduction, each x-coordinate in the positive quadrant will be represented by a trace, and each y-coordinate by a point in time.[2] In order to keep track of which trace represents which x-coordinate, we use one designated atomic proposition x that holds on exactly one time point in each trace: x holds at time i if and only if the trace represents x-coordinate i.

For this purpose, let Ti and τ_0 be given, and define the following formulas[3] over $\mathrm{AP} = \{x\} \cup Ti$:

- Every trace has exactly one point where x holds:
$$\varphi_1 = \forall\pi.\ (\neg x_\pi\ \mathbf{U}(x_\pi \wedge \mathbf{X}\,\mathbf{G}\,\neg x_\pi))$$
- For every $i \in \mathbb{N}$, there is a trace with x in the i-th position:
$$\varphi_2 = (\exists\pi.\ x_\pi) \wedge (\forall\pi_1.\ \exists\pi_2.\ \mathbf{F}(x_{\pi_1} \wedge \mathbf{X}\,x_{\pi_2}))$$
- If two traces represent the same x-coordinate, then they contain the same tiles:
$$\varphi_3 = \forall\pi_1,\pi_2.\ (\mathbf{F}(x_{\pi_1} \wedge x_{\pi_2}) \to \mathbf{G}(\bigwedge_{\tau\in Ti} (\tau_{\pi_1} \leftrightarrow \tau_{\pi_2})))$$
- Every time point in every trace contains exactly one tile:
$$\varphi_4 = \forall\pi.\ \mathbf{G} \bigvee_{\tau\in Ti} (\tau_\pi \wedge \bigwedge_{\tau'\in Ti\setminus\{\tau\}} \neg(\tau')_\pi)$$
- Tiles match vertically:
$$\varphi_5 = \forall\pi.\ \mathbf{G} \bigvee_{\tau\in Ti} (\tau_\pi \wedge \bigvee\nolimits_{\tau'\in\{\tau'\in Ti \mid \tau(north)=\tau'(south)\}} \mathbf{X}(\tau')_\pi)$$
- Tiles match horizontally:
$$\varphi_6 = \forall\pi_1,\pi_2.\ (\mathbf{F}(x_{\pi_1} \wedge \mathbf{X}\,x_{\pi_2}) \to \mathbf{G} \bigvee_{\tau\in Ti} (\tau_{\pi_1} \wedge \bigvee\nolimits_{\tau'\in\{\tau'\in Ti \mid \tau(east)=\tau'(west)\}} (\tau')_{\pi_2}))$$
- Tile τ_0 occurs infinitely often at x-position 0:
$$\varphi_7 = \exists\pi.\ (x_\pi \wedge \mathbf{G}\,\mathbf{F}(\tau_0)_\pi)$$

[2]Note that this means that if we were to visually represent this construction, traces would be arranged vertically.

[3]Technically, the formula we define is not a HyperLTL formula, since it is not in prenex normal form. Nevertheless, it can be trivially transformed into one by moving the quantifiers over conjunctions.

Finally, take $\varphi_{Ti} = \bigwedge_{1 \leq n \leq 7} \varphi_n$. Collectively, subformulas φ_1–φ_3 are satisfied in exactly those sets of traces that can be interpreted as $\mathbb{N} \times \mathbb{N}$. Subformulas φ_4–φ_6 then hold if and only if the $\mathbb{N} \times \mathbb{N}$ grid is correctly tiled with Ti. Subformula φ_7, finally, holds if and only if the tiling uses the tile τ_0 infinitely often at x-coordinate 0. Overall, this means φ_{Ti} is satisfiable if and only if Ti can recurrently tile the positive quadrant.

The Σ^1_1-hardness of HyperLTL satisfiability therefore follows from the Σ^1_1-hardness of the recurring tiling problem [Har85]. □

The Σ^1_1-completeness of HyperLTL satisfiability still holds if we restrict to ultimately periodic traces.

Theorem 3.4. *HyperLTL satisfiability restricted to sets of ultimately periodic traces is Σ^1_1-complete.*

Proof. The problem of whether there is a tiling of $\{(i, j) \in \mathbb{N}^2 \mid i \geq j\}$, i.e. the part of $\mathbb{N} \times \mathbb{N}$ below the diagonal, such that a designated tile τ_0 occurs on every row, is also Σ^1_1-complete [Har85].[4] We reduce this problem to HyperLTL satisfiability on ultimately periodic traces.

The reduction is very similar to the one discussed above, with the necessary changes being: (i) every time point beyond x satisfies the special tile "null", (ii) horizontal and vertical matching are only checked at or before time point x and (iii) for every trace π_1 there is a trace π_2 such that π_2 has designated tile τ_0 at the time where π_1 satisfies x (so τ_0 holds at least once in every row).

Membership in Σ^1_1 can be shown similarly to the proof of 3.2. So, the problem is Σ^1_1-complete. □

Furthermore, a careful analysis of the proof of 3.4 shows that we can restrict ourselves to ultimately periodic traces of the form $x \cdot \emptyset^\omega$, i.e. to essentially finite traces.

Finally, let us also state for completeness the complexity of finite-state satisfiability, i.e., the question whether a given HyperLTL sentence is satisfied by the set of traces of some finite transition system. The result is a direct consequence of HyperLTL model-checking being decidable [FRS15] (i.e., one can exhaustively enumerate and model-check all finite transition systems), yielding the upper bound, and a careful analysis of the undecidability proof for HyperLTL satisfiability due to Finkbeiner and Hahn: they present a reduction from Post's correspondence problem (PCP) to HyperLTL satisfiability so that the following are equivalent for any PCP instance P and resulting HyperLTL sentence φ_P:

- P has a solution.
- φ_P is satisfiable.
- φ_P has a finite model of ultimately periodic traces.

Proposition 3.5 ([FRS15, FH16]). HyperLTL finite-state satisfiability is Σ^0_1-complete.

4. The HyperLTL Quantifier Alternation Hierarchy

The number of quantifier alternations in a formula is a crucial parameter in the complexity of HyperLTL model-checking [FRS15, Rab16]. A natural question is then to understand *which* properties can be expressed with n quantifier alternations, that is, given a sentence φ, determine if there exists an equivalent one with at most n alternations. In this section, we show that this problem is in fact exactly as hard as the HyperLTL unsatisfiability problem (which asks whether a HyperLTL

[4] The proof in [Har85] is for the part *above* the diagonal with τ_0 occurring on every column, but that is easily seen to be equivalent.

sentence has no model), and therefore Π^1_1-complete. Here, Π^1_1 is the co-class of Σ^1_1, i.e. it contains the complements of the Σ^1_1 sets.

4.1. **Definition and strictness of the hierarchy.** Formally, the HyperLTL quantifier alternation hierarchy is defined as follows. Let φ be a HyperLTL formula. We say that φ is a Σ_0- or a Π_0-formula if it is quantifier-free. It is a Σ_n-formula if it is of the form $\varphi = \exists \pi_1. \cdots \exists \pi_k. \psi$ and ψ is a Π_{n-1}-formula. It is a Π_n-formula if it is of the form $\varphi = \forall \pi_1. \cdots \forall \pi_k. \psi$ and ψ is a Σ_{n-1}-formula. We do not require each block of quantifiers to be non-empty, i.e. we may have $k = 0$ and $\varphi = \psi$. Note that formulas in $\Sigma_0 = \Pi_0$ have free variables. As we are only interested in sentences, we disregard $\Sigma_0 = \Pi_0$ in the following and only consider the levels Σ_n and Π_n for $n > 0$.

By a slight abuse of notation, we also let Σ_n denote the set of hyperproperties definable by a Σ_n-sentence, that is, the set of all $L(\varphi) = \{T \subseteq (2^{\mathrm{AP}})^\omega \mid T \models \varphi\}$ such that φ is a Σ_n-sentence of HyperLTL.

Theorem 4.1 ([Rab16, Corollary 5.6.5]). The quantifier alternation hierarchy of HyperLTL is strict: for all $n > 0$, $\Sigma_n \subsetneq \Sigma_{n+1}$.

The strictness of the hierarchy also holds if we restrict our attention to sentences whose models consist of finite sets of traces that end in the suffix $\emptyset^\omega$, i.e. that are essentially finite.

Theorem 4.2. *For all $n > 0$, there exists a Σ_{n+1}-sentence φ of HyperLTL that is not equivalent to any Σ_n-sentence, and such that for all $T \subseteq (2^{\mathrm{AP}})^\omega$, if $T \models \varphi$ then T contains finitely many traces and $T \subseteq (2^{\mathrm{AP}})^* \emptyset^\omega$.*

This property is a necessary ingredient for our argument that membership at some fixed level of the quantifier alternation hierarchy is Π^1_1-hard. It could be derived from a small adaptation of the proof in [Rab16], and we provide for completeness an alternative proof by exhibiting a connection between the HyperLTL quantifier alternation hierarchy and the quantifier alternation hierarchy for first-order logic over finite words, which is known to be strict [CB71, Tho82]. The remainder of the subsection is dedicated to the proof of 4.2.

The proof is organised as follows. We first define an encoding of finite words as sets of traces. We then show that every first-order formula can be translated into an equivalent (modulo encodings) HyperLTL formula with the same quantifier prefix (4.4). Finally, we show how to translate back HyperLTL formulas into FO[$\leq$] formulas with the same quantifier prefix (4.7), so that if the HyperLTL quantifier alternation would hierarchy collapse, then so would the hierarchy for FO[$\leq$].

First-Order Logic over Words. Let AP be a finite set of atomic propositions. A finite word over AP is a finite sequence $w = w(0)w(1)\cdots w(k)$ with $w(i) \in 2^{\mathrm{AP}}$ for all i. We let $|w|$ denote the *length* of w, and $pos(w) = \{0, \ldots, |w| - 1\}$ the set of *positions* of w. The set of all finite words over AP is $(2^{\mathrm{AP}})^*$.

Assume a countably infinite set of variables *Var*. The set of FO[$\leq$] formulas is given by the grammar

$$\varphi ::= a(x) \mid x \leq y \mid \neg\varphi \mid \varphi \vee \varphi \mid \exists x.\ \varphi \mid \forall x.\ \varphi\,,$$

where $a \in \mathrm{AP}$ and $x, y \in Var$. The set of free variables of φ is denoted $\mathrm{Free}(\varphi)$. A sentence is a formula without free variables.

The semantics is defined as follows, $w \in (2^{\mathrm{AP}})^*$ being a finite word and $\nu : \mathrm{Free}(\varphi) \to pos(w)$ an interpretation mapping variables to positions in w:

- $(w, \nu) \models a(x)$ if $a \in w(\nu(x))$.
- $(w, \nu) \models x \leq y$ if $\nu(x) \leq \nu(y)$.
- $(w, \nu) \models \neg\varphi$ if $w, \nu \not\models \varphi$.

- $(w, \nu) \models \varphi \vee \psi$ if $w, \nu \models \varphi$ or $(w, \nu) \models \psi$.
- $(w, \nu) \models \exists x.\ \varphi$ if there exists a position $n \in pos(w)$ such that $(w, \nu[x \mapsto n]) \models \varphi$.
- $(w, \nu) \models \forall x.\ \varphi$ if for all positions $n \in pos(w)$: $(w, \nu[x \mapsto n]) \models \varphi$.

If φ is a sentence, we write $w \models \varphi$ instead of $(w, \nu) \models \varphi$.

As for HyperLTL, a FO[$\leq$] formula in prenex normal form is a Σ_n-formula if its quantifier prefix consists of n alternating blocks of quantifiers (some of which may be empty), starting with a block of existential quantifiers. We let $\Sigma_n(\text{FO}[\leq])$ denote the class of languages of finite words definable by Σ_n-sentences.

Theorem 4.3 ([Tho82, CB71]). The quantifier alternation hierarchy of FO[$\leq$] is strict: for all $n \geq 0$, $\Sigma_n(\text{FO}[\leq]) \subsetneq \Sigma_{n+1}(\text{FO}[\leq])$.

Encodings of Words. The idea to prove 4.2 is to encode a word $w \in (2^{\text{AP}})^*$ as a set of traces T where each trace in T corresponds to a position in w; letters in the word are reflected in the label of the first position of the corresponding trace in T, while the total order $<$ is encoded using a fresh proposition $o \notin \text{AP}$. More precisely, each trace has a unique position labelled o, distinct from one trace to another, and traces are ordered according to the order of appearance of the proposition o. Note that there are several possible encodings for a same word, and we may fix a canonical one when needed. This is defined more formally below.

A *stretch function* is a monotone funtion $f : \mathbb{N} \to \mathbb{N} \setminus \{0\}$, i.e. it satisfies $0 < f(0) < f(1) < \cdots$. For all words $w \in (2^{\text{AP}})^*$ and stretch functions f, we define the set of traces $enc(w, f) = \{t_n \mid n \in pos(w)\} \subseteq (2^{\text{AP} \cup \{o\}})^* \emptyset^\omega$ as follows: for all $i \in \mathbb{N}$,

- for all $a \in \text{AP}$, $a \in t_n(i)$ if and only if $i = 0$ and $a \in w(n)$
- $o \in t_n(i)$ if and only if $i = f(n)$.

It will be convenient to consider encodings with arbitrarily large spacing between o's positions. To this end, for every $N \in \mathbb{N}$, we define a particular encoding

$$enc_N(w) = enc(w, n \mapsto N(n+1))\,.$$

So in $enc_N(w)$, two positions with non-empty labels are at distance at least N from one another.

Given $T = enc(w, f)$ and a trace assignment $\Pi \colon \mathcal{V} \to T$, we let $T^{(N)} = enc_N(w)$, and $\Pi^{(N)} \colon \mathcal{V} \to T^{(N)}$ the trace assignment defined by shifting the o position in each $\Pi(\pi)$ accordingly, i.e.

- $o \in \Pi^{(N)}(\pi)(N(i+1))$ if and only if $o \in \Pi(\pi)(f(i))$ and
- for all $a \in \text{AP}$: $a \in \Pi^N(\pi)(0)$ if and only if $a \in \Pi(\pi)(0)$.

From FO to HyperLTL. We associate with every FO[$\leq$] formula φ in prenex normal form a HyperLTL formula $enc(\varphi)$ over $\text{AP} \cup \{o\}$ by replacing in φ:

- $a(x)$ with a_x, and
- $x \leq y$ with $\mathbf{F}(o_x \wedge \mathbf{F}\, o_y)$.

In particular, $enc(\varphi)$ has the same quantifier prefix as φ, which means that we treat variables of φ as trace variables of $enc(\varphi)$.

Lemma 4.4. *For every* FO[$\leq$] *sentence φ in prenex normal form, φ is equivalent to $enc(\varphi)$ in the following sense: for all $w \in (2^{\text{AP}})^*$ and all stretch functions f,*

$$w \models \varphi \quad \textit{if and only if} \quad enc(w, f) \models enc(\varphi)\,.$$

Proof. By induction over the construction of φ, relying on the fact that traces in $enc(w, f)$ are in bijection with positions in w. □

In particular, note that the evaluation of $enc(\varphi)$ on $enc(w, f)$ does not depend on f. We call such a formula *stretch-invariant*: a HyperLTL sentence φ is *stretch-invariant* if for all finite words w and all stretch functions f and g,

$$enc(w, f) \models \varphi \quad \text{if and only if} \quad enc(w, g) \models \varphi\,.$$

Lemma 4.5. *For all $\varphi \in \mathrm{FO}[\leq]$, $enc(\varphi)$ is stretch-invariant.*

Proof. By induction over the construction of $enc(\varphi)$, relying on the fact that the only temporal subformulas of $enc(\varphi)$ are of the form $\mathbf{F}(o_x \wedge \mathbf{F}\, o_y)$. □

Going Back From HyperLTL to FO. Let $enc(\mathrm{FO}[\leq])$ denote the fragment of HyperLTL consisting of all formulas $enc(\varphi)$, where φ is an $\mathrm{FO}[\leq]$ formula in prenex normal form. Equivalently, $\psi \in enc(\mathrm{FO}[\leq])$ if it is a HyperLTL formula of the form $\psi = Q_1 x_1 \cdots Q_k x_k.\ \psi_0$, where ψ_0 is a Boolean combination of formulas of the form a_x or $\mathbf{F}(o_x \wedge \mathbf{F}\, o_y)$.

Let us prove that every HyperLTL sentence is equivalent, over sets of traces of the form $enc(w, f)$, to a sentence in $enc(\mathrm{FO}[\leq])$ with the same quantifier prefix. This means that if a HyperLTL sentence $enc(\varphi)$ is equivalent to a HyperLTL sentence with a smaller number of quantifier alternations, then it is also equivalent over all word encodings to one of the form $enc(\psi)$, which in turns implies that the $\mathrm{FO}[\leq]$ sentences φ and ψ are equivalent.

The *temporal depth* of a quantifier-free formula in HyperLTL is defined inductively as

- $depth(a_\pi) = 0$,
- $depth(\neg\varphi) = depth(\varphi)$,
- $depth(\varphi \vee \psi) = \max(depth(\varphi), depth(\psi))$,
- $depth(\mathbf{X}\,\varphi) = 1 + depth(\varphi)$, and
- $depth(\varphi\,\mathbf{U}\,\psi) = 1 + \max(depth(\varphi, \psi))$.

For a general HyperLTL formula $\varphi = Q_1\pi_1 \cdots Q_k\pi_k.\ \psi$, we let $depth(\varphi) = depth(\psi)$.

Lemma 4.6. *Let ψ be a quantifier-free formula of HyperLTL. Let $N = depth(\psi) + 1$. There exists a quantifier-free formula $\widehat{\psi} \in enc(\mathrm{FO}[\leq])$ such that for all $T = enc(w, f)$ and trace assignments Π,*

$$(T^{(N)}, \Pi^{(N)}) \models \psi \quad \textit{if and only if} \quad (T, \Pi) \models \widehat{\psi}\,.$$

Proof. Assume that $\mathrm{Free}(\psi) = \{\pi_1, \ldots, \pi_k\}$ is the set of free variables of ψ. Note that the value of $(T^{(N)}, \Pi^{(N)}) \models \psi$ depends only on the traces $\Pi^{(N)}(\pi_1), \ldots, \Pi^{(N)}(\pi_k)$. We see the tuple $(\Pi^{(N)}(\pi_1), \ldots, \Pi^{(N)}(\pi_k))$ as a single trace $w_{T,\Pi,N}$ over the set of propositions $\mathrm{AP}' = \{a_\pi \mid a \in \mathrm{AP} \cup \{o\} \wedge \pi \in \mathrm{Free}(\psi)\}$, and ψ as an LTL formula over AP'.

We are going to show that the evaluation of ψ over words $w_{T,\Pi,N}$ is entirely determined by the ordering of $o_{\pi_1}, \ldots, o_{\pi_n}$ in $w_{T,\Pi,N}$ and the label of $w_{T,\Pi,N}(0)$, which we can both describe using a formula in $enc(\mathrm{FO}[\leq])$. The intuition is that non-empty labels in $w_{T,\Pi,N}$ are at distance at least N from one another, and a temporal formula of depth less than N cannot distinguish between $w_{T,\Pi,N}$ and other words with the same sequence of non-empty labels and sufficient spacing between them. More generally, the following can be easily proved via Ehrenfeucht-Fraïssé games:

Claim 1. Let $m, n \geq 0$, $(a_i)_{i\in\mathbb{N}}$ be a sequence of letters in $2^{\mathrm{AP}'}$, and

$$w_1, w_2 \in \emptyset^m a_0 \emptyset^n \emptyset^* a_1 \emptyset^n \emptyset^* a_2 \emptyset^n \emptyset^* \cdots$$

Then for all LTL formulas φ such that $depth(\varphi) \leq n$, $w_1 \models \varphi$ if and only if $w_2 \models \varphi$.

Here we are interested in words of a particular shape. Let L_N be the set of infinite words $w \in (2^{\mathrm{AP}'})^\omega$ such that:

- For all $\pi \in \{\pi_1, \dots, \pi_k\}$, there is a unique $i \in \mathbb{N}$ such that $o_\pi \in w(i)$. Moreover, $i \geq N$.
- If $o_\pi \in w(i)$ and $o_{\pi'} \in w(i')$, then $|i - i'| \geq N$ or $i = i'$.
- If $a_\pi \in w(i)$ for some $a \in \mathrm{AP}$ and $\pi \in \{\pi_1, \dots, \pi_k\}$, then $i = 0$.

Notice that $w_{T,\Pi,N} \in L_N$ for all T and all Π.

For $w_1, w_2 \in L_N$, we write $w_1 \sim w_2$ if w_1 and w_2 differ only in the spacing between non-empty positions, that is, if there are $\ell \leq k$ and $a_0, \dots, a_\ell \in 2^{\mathrm{AP}'}$ such that $w_1, w_2 \in a_0\emptyset^* a_1 \emptyset^* \cdots a_\ell \emptyset^\omega$. Notice that $\sim$ is of finite index. Moreover, we can distinguish between its equivalence classes using formulas defined as follows. For all $A \subseteq \{a_\pi \mid a \in \mathrm{AP} \wedge \pi \in \{\pi_1, \dots, \pi_k\}\}$ and all total preorders $\preceq$ over $\{\pi_1, \dots, \pi_k\}$,[5] we let

$$\varphi_{A,\preceq} = \bigwedge_{a \in A} a \wedge \bigwedge_{a \notin A} \neg a \wedge \bigwedge_{\pi_i \preceq \pi_j} \mathbf{F}(o_{\pi_i} \wedge \mathbf{F}\, o_{\pi_j})\,.$$

Note that every word $w \in L_N$ satisfies exactly one formula $\varphi_{A,\preceq}$, and that all words in an equivalence class satisfy the same one. We denote by $L_{A,\preceq}$ the equivalence class of $L_N/\!\sim$ consisting of words satisfying $\varphi_{A,\preceq}$. So we have $L_N = \biguplus L_{A,\preceq}$.

Since ψ is of depth less than N, by 1 (with $n = N-1$ and $m = 0$), for all $w_1 \sim w_2$ we have $w_1 \models \psi$ if and only if $w_2 \models \psi$. Now, define $\widehat{\psi}$ as the disjunction of all $\varphi_{A,\preceq}$ such that ψ is satisfied by elements in the class $L_{A,\preceq}$. Then $\widehat{\psi} \in \mathit{enc}(\mathrm{FO}[\leq])$, and

$$\text{for all } w \in L_N, \quad w \models \widehat{\psi} \text{ if and only if } w \models \psi\,.$$

In particular, for every T and every Π, we have $(T^{(N)}, \Pi^{(N)}) \models \widehat{\psi}$ if and only if $(T^{(N)}, \Pi^{(N)}) \models \psi$. Since the preorder between propositions o_π and the label of the initial position are the same in $(T^{(N)}, \Pi^{(N)})$ and (T, Π), we also have $(T, \Pi) \models \widehat{\psi}$ if and only if $(T^{(N)}, \Pi^{(N)}) \models \widehat{\psi}$. Therefore,

$$(T^{(N)}, \Pi^{(N)}) \models \psi \quad \text{if and only if} \quad (T, \Pi) \models \widehat{\psi}\,. \qquad \square$$

For a quantified HyperLTL sentence $\varphi = Q_1\pi_1 \cdots Q_k\pi_k.\ \psi$, we let $\widehat{\varphi} = Q_1\pi_1 \dots Q_k\pi_k.\ \widehat{\psi}$, where $\widehat{\psi}$ is the formula obtained through 4.6.

Lemma 4.7. *For all HyperLTL formulas φ, for all $T = \mathit{enc}(w, f)$ and trace assignments Π,*

$$(T^{(N)}, \Pi^{(N)}) \models \varphi \text{ if and only if } (T, \Pi) \models \widehat{\varphi}\,,$$

where $N = \mathit{depth}(\varphi) + 1$.

Proof. We prove the result by induction. Quantifier-free formulas are covered by 4.6. We have

$$\begin{aligned}
(T, \Pi) \models \exists \pi.\ \widehat{\psi} \quad &\Leftrightarrow \quad \exists t \in T \text{ such that } (T, \Pi[\pi \mapsto t]) \models \widehat{\psi} \\
&\Leftrightarrow \quad \exists t \in T \text{ such that } (T^{(N)}, (\Pi[\pi \mapsto t])^{(N)}) \models \psi \qquad \text{(IH)} \\
&\Leftrightarrow \quad \exists t \in T^{(N)} \text{ such that } (T^{(N)}, \Pi^{(N)}[\pi \mapsto t]) \models \psi \\
&\Leftrightarrow \quad (T^{(N)}, \Pi^{(N)}) \models \exists \pi.\ \psi\,,
\end{aligned}$$

[5] I.e. $\preceq$ is required to be transitive and for all $\pi, \pi' \in \{\pi_1, \dots, \pi_k\}$, we have $\pi \preceq \pi'$ or $\pi' \preceq \pi$ (or both)

and similarly,

$$\begin{aligned}
(T,\Pi) \models \forall\pi.\ \widehat{\psi} \quad &\Leftrightarrow \quad \forall t \in T, \text{we have } (T, \Pi[\pi \mapsto t]) \models \widehat{\psi} \\
&\Leftrightarrow \quad \forall t \in T, \text{we have } (T^{(N)}, (\Pi[\pi \mapsto t])^{(N)}) \models \psi \qquad \text{(IH)} \\
&\Leftrightarrow \quad \forall t \in T^{(N)}, \text{we have } (T^{(N)}, \Pi^{(N)}[\pi \mapsto t]) \models \psi \\
&\Leftrightarrow \quad (T^{(N)}, \Pi^{(N)}) \models \forall\pi.\ \psi\,. \qquad \square
\end{aligned}$$

As a consequence, we obtain the following equivalence.

Lemma 4.8. *For all stretch-invariant HyperLTL sentences φ and for all $T = enc(w, f)$,*

$$T \models \varphi \quad \textit{if and only if} \quad T \models \widehat{\varphi}\,.$$

Proof. By definition of φ being stretch-invariant, we have $T \models \varphi$ if and only if $T^{(N)} \models \varphi$, which by 4.7 is equivalent to $T \models \widehat{\varphi}$. $\square$

We are now ready to prove the strictness of the HyperLTL quantifier alternation hierarchy.

Proof of 4.2. Suppose towards a contradiction that the hierarchy collapses at level $n > 0$, i.e. every HyperLTL Σ_{n+1}-sentence is equivalent to some Σ_n-sentence. Let us show that the FO[$\leq$] quantifier alternation hierarchy also collapses at level n, a contradiction with 4.3.

Fix a Σ_{n+1}-sentence φ of FO[$\leq$]. The HyperLTL sentence $enc(\varphi)$ has the same quantifier prefix as φ, i.e. is also a Σ_{n+1}-sentence. Due to the assumed hierarchy collapse, there exists a HyperLTL Σ_n-sentence ψ that is equivalent to $enc(\varphi)$, and is stretch-invariant by 4.5. Then the HyperLTL sentence $\widehat{\psi}$ defined above is also a Σ_n-sentence. Moreover, since $\widehat{\psi} \in enc(\text{FO}[\leq])$, there exists a FO[$\leq$] sentence φ' such that $\widehat{\psi} = enc(\varphi')$, which has the same quantifier prefix as $\widehat{\psi}$, i.e. φ' is a Σ_n-sentence of FO[$\leq$]. For all words $w \in (2^{\text{AP}})^*$, we now have

$$\begin{aligned}
w \models \varphi \quad &\text{if and only if} \quad enc(w,f) \models enc(\varphi) && (\ 4.4) \\
&\text{if and only if} \quad enc(w,f) \models \psi && \text{(assumption)} \\
&\text{if and only if} \quad enc(w,f) \models \widehat{\psi} && (\ 4.8 \text{ and } 4.5) \\
&\text{if and only if} \quad enc(w,f) \models enc(\varphi') && \text{(definition)} \\
&\text{if and only if} \quad w \models \varphi' && (\ 4.4)
\end{aligned}$$

for an arbitrary stretch function f. Therefore, $\Sigma_{n+1}(\text{FO}[\leq]) = \Sigma_n(\text{FO}[\leq])$, yielding the desired contradiction.

This proves not only that for all $n > 0$, there is a HyperLTL Σ_{n+1}-sentence that is not equivalent to any Σ_n-sentence, but also that there is one of the form $enc(\varphi)$. Now, the proof still goes through if we replace $enc(\varphi)$ by any formula equivalent to $enc(\varphi)$ over all $enc(w, f)$, and in particular if we replace $enc(\varphi)$ by $enc(\varphi) \wedge \psi$, where the sentence

$$\psi = \exists\pi.\ \forall\pi'.\ (\mathbf{F}\,\mathbf{G}\,\emptyset_\pi) \wedge \mathbf{G}(\mathbf{G}\,\emptyset_\pi \rightarrow \mathbf{G}\,\emptyset_{\pi'})$$

with $\emptyset_\pi = \bigwedge_{a \in \text{AP}} \neg a_\pi$ selects models that contain finitely many traces, all in $(2^{\text{AP}})^* \cdot \emptyset^\omega$. Indeed, all $enc(w, f)$ satisfy ψ. Notice that ψ is a Σ_2-sentence, and since $n + 1 \geq 2$, (the prenex normal form of) $enc(\varphi) \wedge \psi$ is still a Σ_{n+1}-sentence. $\square$

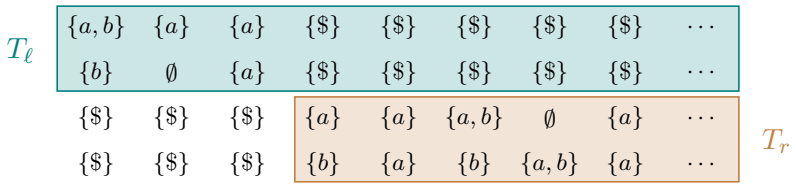


FIGURE 1. Example of a split set of traces where each row represents a trace and $b = 3$.

4.2. **Membership problem.** In this subsection, we investigate the complexity of the membership problem for the HyperLTL quantifier alternation hierarchy. Our goal is to prove the following result.

Theorem 4.9. *Fix $n > 0$. The problem of deciding whether a given HyperLTL sentence is equivalent to some Σ_n-sentence is Π^1_1-complete.*

The easier part of the proof will be the upper bound, since a corollary of 3.1 is that the problem of deciding whether two HyperLTL formulas are equivalent is Π^1_1-complete.

The lower bound will be proven by a reduction from the HyperLTL unsatisfiability problem. The proof relies on 4.2: given a sentence φ, we are going to combine φ with some Σ_{n+1}-sentence φ_{n+1} witnessing the strictness of the hierarchy, to construct a sentence ψ such that φ is unsatisfiable if and only if ψ is equivalent to a Σ_n-sentence. Intuitively, the formula ψ will describe models consisting of the "disjoint union" of a model of φ_{n+1} and a model of φ. Here "disjoint" is to be understood in a strong sense: we split both the set of traces and the time domain into two parts, used respectively to encode the models of φ_{n+1} and those of φ.

To make this more precise, let us introduce some notations. We assume a distinguished symbol $\$ \notin \mathrm{AP}$. We say that a set of traces $T \subseteq (2^{\mathrm{AP}\cup\{\$\}})^\omega$ is *bounded* if there exists $b \in \mathbb{N}$ such that $T \subseteq (2^{\mathrm{AP}})^b \cdot \{\$\}^\omega$.

Lemma 4.10. *There exists a Π_1-sentence φ_{bd} such that for all $T \subseteq (2^{\mathrm{AP}\cup\{\$\}})^\omega$, we have $T \models \varphi_{bd}$ if and only if T is bounded.*

Proof. We let

$$\varphi_{bd} = \forall \pi, \pi'.\ (\neg \$_\pi \,\mathbf{U}\, \mathbf{G}\, \$_\pi) \wedge \bigwedge_{a\in\mathrm{AP}} \mathbf{G}(\neg(a_\pi \wedge \$_\pi)) \wedge \mathbf{F}\,(\neg\$_\pi \wedge \neg\$_{\pi'} \wedge \mathbf{X}\,\$_\pi \wedge \mathbf{X}\,\$_{\pi'})\ .$$

The conjunct $(\neg \$_\pi \,\mathbf{U}\, \mathbf{G}\, \$_\pi) \wedge \bigwedge_{a\in\mathrm{AP}} \mathbf{G}(\neg(a_\pi \wedge \$_\pi))$ ensures that every trace is in $(2^{\mathrm{AP}})^* \cdot \{\$\}^\omega$, while $\mathbf{F}\,(\neg\$_\pi \wedge \neg\$_{\pi'} \wedge \mathbf{X}\,\$_\pi \wedge \mathbf{X}\,\$_{\pi'})$ ensures that the \$'s in any two traces π and π' start at the same position. □

We say that a nonempty set T of traces is *split* if there exist a $b \in \mathbb{N}$ and T_1, T_2 such that $T = T_1 \uplus T_2$, $T_1 \subseteq (2^{\mathrm{AP}})^b \cdot \{\$\}^\omega$, and $T_2 \subseteq \{\$\}^b \cdot (2^{\mathrm{AP}})^\omega$. Note that b as well as T_1 and T_2 are unique then. Hence, we define the left and right part of T as $T_\ell = T_1$ and $T_r = \{t \in (2^{\mathrm{AP}})^\omega \mid \{\$\}^b \cdot t \in T_2\}$, respectively (see Figure 1).

It is easy to combine HyperLTL specifications for the left and right part of a split model into one global formula.

Lemma 4.11. *For all HyperLTL sentences φ_ℓ, φ_r, one can construct a sentence ψ such that for all split $T \subseteq (2^{\mathrm{AP}\cup\{\$\}})^\omega$, it holds that $T_\ell \models \varphi_\ell$ and $T_r \models \varphi_r$ if and only if $T \models \psi$.*

Proof. Let $\widehat{\varphi_r}$ denote the formula obtained from φ_r by replacing:

- every existential quantification $\exists \pi.\ \varphi$ with $\exists \pi.\ ((\mathbf{F}\,\mathbf{G}\,\neg\$_\pi) \wedge \varphi)$;

- every universal quantification $\forall\pi.\ \varphi$ with $\forall\pi.\ ((\mathbf{F}\,\mathbf{G}\,\neg\$_\pi) \to \varphi)$;
- the quantifier-free part φ of φ_r with $\$_\pi\,\mathbf{U}(\neg\$_\pi \wedge \varphi)$, where π is some free variable in φ.

Here, the first two replacements restrict quantification to traces in the right part while the last one requires the formula to hold at the first position of the right part. We define $\widehat{\varphi_\ell}$ by similarly relativizing quantifications in φ_ℓ. The formula $\widehat{\varphi_\ell} \wedge \widehat{\varphi_r}$ can then be put back into prenex normal form to define ψ. □

Conversely, any HyperLTL formula that only has split models can be decomposed into a Boolean combination of formulas that only talk about the left or right part of the model. This is formalised in the lemma below.

Lemma 4.12. *For all HyperLTL Σ_n-sentences φ there exists a finite family $(\varphi_\ell^i, \varphi_r^i)_i$ of Σ_n-sentences such that for all split $T \subseteq (2^{\mathrm{AP}\cup\{\$\}})^\omega$: $T \models \varphi$ if and only if there is an i with $T_\ell \models \varphi_\ell^i$ and $T_r \models \varphi_r^i$.*

Proof. To prove this result by induction, we need to strengthen the statement to make it dual and allow for formulas with free variables. We let $\mathrm{Free}(\varphi)$ denote the set of free variables of a formula φ. We prove the following result, which implies 4.12.

Claim 2. For all HyperLTL Σ_n-formulas (resp. Π_n-formulas) φ, there exists a finite family of Σ_n-formulas (resp. Π_n-formulas) $(\varphi_\ell^i, \varphi_r^i)_i$ such that for all i, $\mathrm{Free}(\varphi) = \mathrm{Free}(\varphi_\ell^i) \uplus \mathrm{Free}(\varphi_r^i)$, and for all split T and Π: $(T, \Pi) \models \varphi$ if and only if there exists i such that

- For all $\pi \in \mathrm{Free}(\varphi)$, $\Pi(\pi) \in T_\ell$ if and only if $\pi \in \mathrm{Free}(\varphi_\ell^i)$ (and thus $\Pi(\pi) \in T \setminus T_\ell$ if and only if $\pi \in \mathrm{Free}(\varphi_r^i)$).
- $(T_\ell, \Pi) \models \varphi_\ell^i$;
- $(T_r, \Pi') \models \varphi_r^i$, where Π' maps every $\pi \in \mathrm{Free}(\varphi_r^i)$ to the trace in T_r corresponding to $\Pi(\pi)$ in T (i.e. $\Pi(\pi) = \{\$\}^b \cdot \Pi'(\pi)$ for some b).

To simplify, we can assume that the partition of the free variables of φ into a left and right part is fixed, i.e. we take $V_\ell \subseteq \mathrm{Free}(\varphi)$ and $V_r = \mathrm{Free}(\varphi) \setminus V_\ell$, and we restrict our attention to split T and Π such that $\Pi(V_\ell) \subseteq T_\ell$ and $\Pi(V_r) \subseteq T \setminus T_\ell$. The formulas $(\varphi_\ell^i, \varphi_r^i)_i$ we are looking for should then be such that $\mathrm{Free}(\varphi_\ell^i) = V_\ell$ and $\mathrm{Free}(\varphi_r^i) = V_r$. If we can define sets of formulas $(\varphi_\ell^i, \varphi_r^i)$ for each choice of V_ℓ, V_r, then the general case is solved by taking the union of all of those. So we focus on a fixed V_ℓ, V_r, and prove the result by induction on the quantifier depth of φ.

Base case. If φ is quantifier-free, then it can be seen as an LTL formula over the set of propositions $\{a_\pi, \$_\pi \mid \pi \in \mathrm{Free}(\varphi), a \in \mathrm{AP}\}$, and any split model of φ consistent with V_ℓ, V_r can be seen as a word in $\Sigma_\ell^* \cdot \Sigma_r^\omega$, where

$$\Sigma_\ell = \left\{\alpha \cup \{\$_\pi \mid \pi \in V_r\} \mid \alpha \subseteq \{a_\pi \mid \pi \in V_\ell \wedge a \in \mathrm{AP}\}\right\} \text{ and}$$
$$\Sigma_r = \left\{\alpha \cup \{\$_\pi \mid \pi \in V_\ell\} \mid \alpha \subseteq \{a_\pi \mid \pi \in V_r \wedge a \in \mathrm{AP}\}\right\}.$$

Note in particular that $\Sigma_\ell \cap \Sigma_r = \emptyset$. We can thus conclude by applying the following standard result of formal language theory:

Claim 3. Let $L \subseteq \Sigma_1^* \cdot \Sigma_2^\omega$, where $\Sigma_1 \cap \Sigma_2 = \emptyset$. If $L = L(\varphi)$ for some LTL formula φ, then there exists a finite family $(\varphi_1^i, \varphi_2^i)_i$ of LTL formulas such that $L = \bigcup_{1 \le i \le k} L(\varphi_1^i) \cdot L(\varphi_2^i)$ and for all i, $L(\varphi_1^i) \subseteq \Sigma_1^*$ and $L(\varphi_2^i) \subseteq \Sigma_2^\omega$.

Proof. A language is definable in LTL if and only if it is accepted by some counter-free automaton [DG08, Tho81]. Let $\mathcal{A}$ be a counter-free automaton for L. For

every state q in $\mathcal{A}$, let

$$L_1^q = \{w \in \Sigma_1^* \mid q_0 \xrightarrow{w} q \text{ for some initial state } q_0\} \text{ and}$$
$$L_2^q = \{w \in \Sigma_2^\omega \mid \text{there is an accepting run on } w \text{ starting from } q\}.$$

We have $L = \bigcup_q L_1^q \cdot L_2^q$. Moreover, L_1^q and L_2^q are still recognisable by counter-free automata, and therefore LTL definable. □

Case $\varphi = \exists \pi.\ \psi$. Let $(\psi^i_{\ell,1}, \psi^i_{r,1})$ and $(\psi^i_{\ell,2}, \psi^i_{r,2})$ be the formulas constructed respectively for $(\psi, V_\ell \cup \{\pi\}, V_r)$ and $(\psi, V_\ell, V_r \cup \{\pi\})$. We take the union of all $(\exists \pi.\ \psi^i_{\ell,1}, \psi^i_{r,1})$ and $(\psi^i_{\ell,2}, \exists \pi.\ \psi^i_{r,2})$.
Case $\varphi = \forall \pi.\ \psi$. Let $(\xi^i_\ell, \xi^i_r)_{1 \le i \le k}$ be the formulas obtained for $\exists \pi.\ \neg\psi$. We have $(T, \Pi) \models \varphi$ if and only if for all i, $(T_\ell, \Pi) \not\models \xi^i_\ell$ or $(T_r, \Pi') \not\models \xi^i_r$; or, equivalently, if there exists $h : \{1, \dots, k\} \to \{\ell, r\}$ such that $(T_\ell, \Pi) \models \bigwedge_{h(i)=\ell} \neg\xi^i_\ell$ and $(T_r, \Pi') \models \bigwedge_{h(i)=r} \neg\xi^i_r$. Take the family $(\varphi^h_\ell, \varphi^h_r)_h$, where $\varphi^h_\ell = \bigwedge_{h(i)=\ell} \neg\xi^i_\ell$ and $\varphi^h_r = \bigwedge_{h(i)=r} \neg\xi^i_r$. Since $\varphi = \forall \pi.\ \psi$ is a Π_n-formula, the formula $\exists \pi.\ \neg\psi$ and by induction all ξ^i_ℓ and ξ^i_r are Σ_n-formulas. Then all $\neg\xi^i_r$ are Π_n-formulas, and since Π_n-formulas are closed under conjunction (up to formula equivalence), all φ^h_ℓ and φ^h_r are Π_n-formulas as well. □

We are now ready to prove 4.9.

Proof of 4.9. The upper bound is an easy consequence of 3.1: Given a HyperLTL sentence φ, we express the existence of a Σ_n-sentence ψ using first-order quantification and encode equivalence of ψ and φ via the formula $(\neg\varphi \wedge \psi) \vee (\varphi \wedge \neg\psi)$, which is unsatisfiable if and only if φ and ψ are equivalent. Altogether, this shows membership in Π^1_1, as Π^1_1 is closed under existential first-order quantification (see, e.g. [Hin17, Page 82]).

We prove the lower bound by reduction from the unsatisfiability problem for HyperLTL. So given a HyperLTL sentence φ, we want to construct ψ such that φ is unsatisfiable if and only if ψ is equivalent to a Σ_n-sentence.

We first consider the case $n > 1$. Fix a Σ_{n+1}-sentence φ_{n+1} that is in not equivalent to any Σ_n-sentence, and such that every model of φ_{n+1} is bounded. The existence of such a formula is a consequence of 4.2: There we have constructed φ_{n+1} such that it only has finite models where every trace in the model ends in $\emptyset^\omega$. The construction can easily be adapted so that φ_{n+1} only has finite models where every trace in the model ends in $\emptyset^\omega$, i.e., it is bounded. Now, by 4.11, there exists a computable ψ such that for all split models T, we have $T \models \psi$ if and only if $T_\ell \models \varphi_{n+1}$ and $T_r \models \varphi$.

First, it is clear that if φ is unsatisfiable, then ψ is unsatisfiable as well, and thus equivalent to $\exists \pi.\ a_\pi \wedge \neg a_\pi$, which is a Σ_n-sentence since $n \ge 1$.

Conversely, suppose towards a contradiction that φ is satisfiable and that ψ is equivalent to some Σ_n-sentence. Let $(\psi^i_\ell, \psi^i_r)_i$ be the finite family of Σ_n-sentences given by 4.12 for ψ. Fix a model T_φ of φ. For a bounded T, we let $\overline{T}$ denote the unique split set of traces such that $\overline{T_\ell} = T$ and $\overline{T_r} = T_\varphi$. For all T, we then have $T \models \varphi_{n+1}$ if and only if T is bounded and $\overline{T} \models \psi$. Recall that the set of bounded models can be defined by a Π_1-sentence φ_{bd} (4.10), which is also a Σ_n-sentence since $n > 1$. We then have $T \models \varphi_{n+1}$ if and only if $T \models \varphi_{bd}$ and there exists i such that $T \models \psi^i_\ell$ and $T_\varphi \models \psi^i_r$. So φ_{n+1} is equivalent to

$$\varphi_{bd} \wedge \bigvee_{i \text{ with } T_\varphi \models \psi^i_r} \psi^i_\ell,$$

which, since Σ_n-sentences are closed (up to logical equivalence) under conjunction and disjunction, is equivalent to a Σ_n-sentence. This contradicts the definition of φ_{n+1}.

We are left with the case $n = 1$. Similarly, we construct ψ such that φ is unsatisfiable if and only if ψ is unsatisfiable, and if and only if ψ is equivalent to a Σ_1-sentence. However, we do not need to use bounded or split models here. Every satisfiable Σ_1-sentence has a model with finitely many traces. Therefore, a simple way to construct ψ so that it is not equivalent to any Σ_1-sentence (unless it is unsatisfiable) is to ensure that every model of ψ contains infinitely many traces.

Let $x \notin \mathrm{AP}$, and $T_\omega = \{\emptyset^n\{x\}\emptyset^\omega \mid n \in \mathbb{N}\}$. As seen in the proof of 3.3, T_ω is definable in HyperLTL: There is a sentence φ_ω such that $T \subseteq (2^{\mathrm{AP}\cup\{x\}})^\omega$ is a model of φ_ω if and only if $T = T_\omega$. By relativising quantifiers in φ_ω and φ to traces with or without the atomic proposition x, one can construct a HyperLTL sentence ψ such that $T \models \psi$ if and only if $T_\omega \subseteq T$ and $T \setminus T_\omega \models \varphi$.

Again, if φ is unsatisfiable then ψ is unsatisfiable and therefore equivalent to $\exists\pi.\ a_\pi \wedge \neg a_\pi$, a Σ_1-sentence. Conversely, all models of ψ contain infinitely many traces and therefore, if ψ is equivalent to a Σ_1-sentence then it is unsatisfiable, and so is φ. □

5. HyperCTL* satisfiability is Σ^2_1-complete

Here, we consider the HyperCTL* satisfiability problem: given a HyperCTL* sentence, determine whether it has a model $\mathcal{T}$ (of arbitrary size). We prove that it is much harder than HyperLTL satisfiability. As a key step of the proof, which is interesting in its own right, we also prove that every satisfiable sentence admits a model of cardinality at most $\mathfrak{c}$ (the cardinality of the continuum). Conversely, we exhibit a satisfiable HyperCTL* sentence whose models are all of cardinality at least $\mathfrak{c}$.

Theorem 5.1. *HyperCTL* satisfiability is Σ^2_1-complete.*

The proof proceeds as follows: In subsection 5.1, we present the upper bound on the size of models of HyperCTL* sentences, which is then used in subsection 5.2 to prove that HyperCTL* membership is in Σ^2_1: Intuitively, the existence of a transition system of cardinality $\mathfrak{c}$ and the fact that it is indeed a model can be expressed using existential quantification of type 2 objects. Finally, in subsection 5.3 we present the matching lower bounds. To this end, we first show that there is a satisfiable HyperCTL* sentence whose models have to contain pairwise disjoint paths encoding all traces over some fixed AP. Note that this formula has only models of cardinality $\mathfrak{c}$, thereby giving a matching lower bound to the upper bound on the size of models of HyperCTL*. Using this model, we can reduce existential third-order arithmetic to HyperCTL* satisfiability.

5.1. An Upper Bound on the Size of HyperCTL* Models.

Before we begin proving membership in Σ^2_1, we obtain a bound on the size of minimal models of satisfiable HyperCTL* sentences. For this, we use an argument based on Skolem functions, which is a transfinite generalisation of the proof that all satisfiable HyperLTL sentences have a countable model [FZ17]. Later, we complement this upper bound by a matching lower bound, which will be applied in the hardness proof.

In the following, we use ω and ω_1 to denote the first infinite and the first uncountable ordinal, respectively, and write $\aleph_0$ and $\aleph_1$ for their cardinality. As $\mathfrak{c}$ is uncountable, we have $\aleph_1 \leq \mathfrak{c}$.

Lemma 5.2. *Each satisfiable HyperCTL* sentence φ has a model of size at most $\mathfrak{c}$.*

The proof of 5.2 uses a Skolem function to create a model. Before giving this proof, we should therefore first introduce Skolem functions for HyperCTL*.

Let φ be a HyperCTL* formula. A quantifier in φ occurs *with polarity 0* if it occurs inside the scope of an even number of negations, and *with polarity 1* if it occurs inside the scope of an odd number of negations. We then say that a quantifier *occurs existentially* if it is an existential quantifier with polarity 0, or a universal quantifier with polarity 1. Otherwise the quantifier *occurs universally*. A Skolem function will map choices for the universally occurring quantifiers to choices for the existentially occurring quantifiers.

For reasons of ease of notation, it is convenient to consider a single Skolem function for all existentially occurring quantifiers in a HyperCTL* formula φ, so the output of the function is an l-tuple of paths, where l is the number of existentially occurring quantifiers in φ. The input consists of a k-tuple of paths, where k is the number of universally occurring quantifiers in φ, plus an l-tuple of integers. The reason for these integers is that we need to keep track of the time point in which the existentially occurring quantifiers are invoked.

Consider, for example, a HyperCTL* formula of the form $\forall \pi_1.\ \mathbf{G}\, \exists \pi_2.\ \psi$. This formula states that for every path π_1, and for every future point $\pi_1(i)$ on that path, there is some π_2 starting in $\pi_1(i)$ satisfying ψ. So the choice of π_2 depends not only on π_1, but also on i. For each existentially occurring quantifier, we need one integer to represent this time point at which it is invoked. A HyperCTL* Skolem function for a formula φ on a transition system $\mathcal{T}$ is therefore a function f of the form $f\colon \mathit{paths}(\mathcal{T})^k \times \mathbb{N}^l \to \mathit{paths}(\mathcal{T})^l$, where $\mathit{paths}(\mathcal{T})$ is the set of paths over $\mathcal{T}$, k is the number of universally occurring quantifiers in φ and l is the number of existentially occurring quantifiers. Note that not every function of this form is a Skolem function, but for our upper bound it suffices that every Skolem function is of that form.

Now, we are able to prove that every satisfiable HyperCTL* formula has a model of size $\mathfrak{c}$.

Proof of 5.2. If φ is satisfiable, let $\mathcal{T}$ be one of its models, and let f be a Skolem function witnessing the satisfaction of φ on $\mathcal{T}$. We create a sequence of transition systems $\mathcal{T}_\alpha$ as follows.

- $\mathcal{T}_0$ contains the vertices and edges of a single, arbitrarily chosen, path of $\mathcal{T}$ starting in the initial vertex.
- $\mathcal{T}_{\alpha+1}$ contains exactly those vertices and edges from $\mathcal{T}$ that are (i) part of $\mathcal{T}_\alpha$ or (ii) among the outputs of the Skolem function f when restricted to input paths from $\mathcal{T}_\alpha$.
- if α is a limit ordinal, then $\mathcal{T}_\alpha = \bigcup_{\alpha'<\alpha} \mathcal{T}_{\alpha'}$.

Note that if α is a limit ordinal then $\mathcal{T}_\alpha$ may contain paths $\rho(0)\rho(1)\rho(2)\cdots$ that are not included in any $\mathcal{T}_{\alpha'}$ with $\alpha' < \alpha$, as long as each finite prefix $\rho(0)\cdots\rho(i)$ is included in some $\alpha'_i < \alpha$.

First, we show that this procedure reaches a fixed point at $\alpha = \omega_1$. Suppose towards a contradiction that $\mathcal{T}_{\omega_1+1} \neq \mathcal{T}_{\omega_1}$. Then there are $\vec{\rho} = (\rho_1, \ldots, \rho_k) \in \mathit{paths}(\mathcal{T}_{\omega_1})^k$ and $\vec{n} \in \mathbb{N}^l$ such that $f(\vec{\rho}, \vec{n}) \notin \mathit{paths}(\mathcal{T}_{\omega_1})^l$. Then for every $i \in \mathbb{N}$ and every $1 \leq j \leq k$, there is an ordinal $\alpha_{i,j} < \omega_1$ such that the finite prefix $\rho_j(0)\cdots\rho_j(i)$ is contained in $\mathcal{T}_{\alpha_{i,j}}$. The set $\{\alpha_{i,j} \mid i \in \mathbb{N}, 1 \leq j \leq k\}$ is countable, and because $\alpha_{i,j} < \omega_1$ each $\alpha_{i,j}$ is also countable. A countable union of countable sets is itself countable, so $\sup\{\alpha_{i,j} \mid i \in \mathbb{N}, 1 \leq j \leq k\} = \bigcup_{i\in\mathbb{N}} \bigcup_{1\leq j\leq k} \alpha_{i,j} = \beta < \omega_1$.

But then the $\vec{\rho}$ are all contained in $\mathcal{T}_\beta$, and therefore $f(\vec{\rho}, \vec{n}) \in \mathit{paths}(\mathcal{T}_{\beta+1})^l$. But $\beta + 1 < \omega_1$, so this contradicts the assumption that $f(\vec{\rho}, \vec{n}) \notin \mathit{paths}(\mathcal{T}_{\omega_1})^l$.

From this contradiction we obtain $\mathcal{T}_{\omega_1+1} = \mathcal{T}_{\omega_1}$, so we have reached a fixed point. Furthermore, because $\mathcal{T}_{\omega_1}$ is contained in $\mathcal{T}$ and closed under the Skolem function and $\mathcal{T}$ satisfies φ, we obtain that $\mathcal{T}_{\omega_1}$ also satisfies φ.

Left to do, then, is to bound the size of $\mathcal{T}_{\omega_1}$, by bounding the number of vertices that get added at each step in its construction. We show by induction that $|\mathcal{T}_\alpha| \leq \mathfrak{c}$ for every α. So, in particular, we have $\mathcal{T}_{\omega_1} \leq \mathfrak{c}$, as required.

As base case, we have $|\mathcal{T}_0| \leq \aleph_0 < \mathfrak{c}$, since it consists the vertices of a single path. Consider then $|\mathcal{T}_{\alpha+1}|$. For each possible input to f, there are at most l new paths, and therefore at most $|\mathbb{N} \times l|$ new vertices in $\mathcal{T}_{\alpha+1}$. Further, there are $|\mathit{paths}(\mathcal{T}_\alpha)|^k \times |\mathbb{N}|^l$ such inputs. By the induction hypothesis, $|\mathcal{T}_\alpha| \leq \mathfrak{c}$, which implies that $|\mathit{paths}(\mathcal{T}_\alpha)| \leq \mathfrak{c}$. As such, the number of added vertices in each step is limited to $\mathfrak{c}^k \times \aleph_0^l \times \aleph_0 \times l = \mathfrak{c}$. So $|\mathcal{T}_{\alpha+1}| \leq |\mathcal{T}_\alpha| + \mathfrak{c} = \mathfrak{c}$.

If α is a limit ordinal, $\mathcal{T}_\alpha$ is a union of at most $\aleph_1$ sets, each of which has, by the induction hypothesis, a size of at most $\mathfrak{c}$. Hence $|\mathcal{T}_\alpha| \leq \aleph_1 \times \mathfrak{c} = \mathfrak{c}$. □

5.2. **HyperCTL* satisfiability is in Σ_1^2.** With the upper bound on the size of models at hand, we can place HyperCTL* satisfiability in Σ_1^2, as the existence of a model of size $\mathfrak{c}$ can be captured by quantification over type 2 objects.

Lemma 5.3. *HyperCTL* satisfiability is in Σ_1^2.*

Proof. As every HyperCTL* formula is satisfied in a model of size at most $\mathfrak{c}$, these models can be represented by objects of type 2. Checking whether a formula is satisfied in a transition system is equivalent to the existence of a winning strategy for Verifier in the induced model checking game. Such a strategy is again a type 2 object, which is existentially quantified. Finally, whether it is winning can be expressed by quantification over individual elements and paths, which are objects of types 0 and 1. Checking the satisfiability of a HyperCTL* formula φ therefore amounts to existential third-order quantification (to choose a model and a strategy) followed by a second-order formula to verify that φ holds on the model (i.e. that the chosen strategy is winning). Hence HyperCTL* satisfiability is in Σ_1^2.

Formally, we encode the existence of a winning strategy for Verifier in the HyperCTL* model checking game $\mathcal{G}(\mathcal{T}, \varphi)$ induced by a transition system $\mathcal{T}$ and a HyperCTL* sentence φ. This game is played between Verifier and Falsifier, one of them aiming to prove that $\mathcal{T} \models \varphi$ and the other aiming to prove $\mathcal{T} \not\models \varphi$. It is played in a graph whose positions correspond to subformulas which they want to check (and suitable path assignments of the free variables): each vertex (say, representing a subformula ψ) belongs to one of the players who has to pick a successor, which represents a subformula of ψ. A play ends at an atomic proposition, at which point the winner can be determined.

Formally, a vertex of the game is of the form (Π, ψ, b) where Π is a path assignment, ψ is a subformula of φ, and $b \in \{0, 1\}$ is a flag used to count (modulo two) the number of negations encountered along the play; the initial vertex is $(\Pi_\emptyset, \varphi, 0)$. Furthermore, for until-subformulas ψ, we need auxiliary vertices of the form (Π, ψ, b, j) with $j \in \mathbb{N}$. The vertices of Verifier are

- of the form $(\Pi, \psi, 0)$ with $\psi = \psi_1 \vee \psi_2$, $\psi = \psi_1 \,\mathbf{U}\, \psi_2$, or $\psi = \exists \pi.\ \psi'$,
- of the form $(\Pi, \forall \pi.\ \psi', 1)$, or
- of the form $(\Pi, \psi_1 \,\mathbf{U}\, \psi_2, 1, j)$.

The moves of the game are defined as follows:

- A vertex (Π, a_π, b) is terminal. It is winning for Verifier if $b = 0$ and $a \in \lambda(\Pi(\pi)(0))$ or if $b = 1$ and $a \notin \lambda(\Pi(\pi)(0))$, where λ is the labelling function of $\mathcal{T}$.
- A vertex $(\Pi, \neg\psi, b)$ has a unique successor $(\Pi, \psi, b + 1 \bmod 2)$.
- A vertex $(\Pi, \psi_1 \vee \psi_2, b)$ has two successors of the form (Π, ψ_i, b) for $i \in \{1, 2\}$.

- A vertex $(\Pi, \mathbf{X}\,\psi, b)$ has a unique successor $(\Pi[1,\infty), \psi, b)$.
- A vertex $(\Pi, \psi_1 \,\mathbf{U}\, \psi_2, b)$ has a successor $(\Pi, \psi_1 \,\mathbf{U}\, \psi_2, b, j)$ for every $j \in \mathbb{N}$.
- A vertex $(\Pi, \psi_1 \,\mathbf{U}\, \psi_2, b, j)$ has the successor $(\Pi[j,\infty), \psi_2, b)$ as well as for every $0 \leq j' < j$ the successor $(\Pi[j',\infty), \psi_1, b)$.
- A vertex $(\Pi, \exists\pi.\ \psi, b)$ has successors $(\Pi[\pi \mapsto \rho], \psi, b)$ for every path ρ of $\mathcal{T}$ starting in $\mathrm{rcnt}(\Pi)$.
- A vertex $(\Pi, \forall\pi.\ \psi, b)$ has successors $(\Pi[\pi \mapsto \rho], \psi, b)$ for every path ρ of $\mathcal{T}$ starting in $\mathrm{rcnt}(\Pi)$.

A play of the model checking game is a finite path through the graph, starting at the initial vertex and ending at a terminal vertex. It is winning for Verifier if the terminal vertex is winning for her. Note that the length of a play is bounded by $2d$, where d is the depth[6] of φ, as the formula is simplified during at least every other move.

A strategy σ for Verifier is a function mapping each of her vertices v to some successor of v. A play $v_0 \cdots v_k$ is consistent with σ, if $v_{k'+1} = \sigma(v_{k'})$ for every $0 \leq k' < k$ such that $v_{k'}$ is a vertex of Verifier. A straightforward induction shows that Verifier has a winning strategy for $\mathcal{G}(\mathcal{T}, \varphi)$ if and only if $\mathcal{T} \models \varphi$.

Recall that every satisfiable HyperCTL* sentence has a model of cardinality $\mathfrak{c}$ (see 5.2). Thus, to place HyperCTL* satisfiability in Σ^2_1, we express, for a given natural number encoding a HyperCTL* formula φ, the existence of the following type 2 objects (using suitable encodings):

- A transition system $\mathcal{T}$ of cardinality $\mathfrak{c}$.
- A function σ from V to V, where V is the set of vertices of $\mathcal{G}(\mathcal{T}, \varphi)$. Note that a single vertex of V is a type 1 object.

Then, we express that σ is a strategy for Verifier, which is easily expressible using quantification over type 1 objects. Thus, it remains to express that σ is winning by stating that every play (a sequence of type 1 objects of bounded length) that is consistent with σ ends in a terminal vertex that is winning for Verifier. Again, we leave the tedious, but standard, details to the reader. □

5.3. **HyperCTL* satisfiability is Σ^2_1-hard.** Next, we prove a matching lower bound. We first describe a satisfiable HyperCTL* sentence $\varphi_{\mathfrak{c}}$ that does not have any model of cardinality less than $\mathfrak{c}$ (more precisely, the initial vertex must have uncountably many successors), thus matching the upper bound from 5.2. We construct $\varphi_{\mathfrak{c}}$ with one particular model $\mathcal{T}_{\mathfrak{c}}$ in mind, defined below, though it also has other models.

The idea is that we want all possible subsets of $A \subseteq \mathbb{N}$ to be represented in $\mathcal{T}_{\mathfrak{c}}$ in the form of paths ρ_A such that $\rho_A(i)$ is labelled by 1 if $i \in A$, and by 0 otherwise. By ensuring that the first vertices of these paths are pairwise distinct, we obtain the desired lower bound on the cardinality. We express this in HyperCTL* as follows: First, we express that there is a part of the model (labelled by `fbt`) where every reachable vertex has two successors, one labelled with 0 and one labelled with 1, i.e. the unravelling of this part contains the full binary tree. Thus, this part has a path ρ_A as above for every subset A, but their initial vertices are not necessarily distinct. Hence, we also express that there is another part (labelled by `set`) that contains a copy of each path in the `fbt`-part, and that these paths indeed start at distinct successors of the initial vertex.

We let $\mathcal{T}_{\mathfrak{c}} = (V_{\mathfrak{c}}, E_{\mathfrak{c}}, t_\varepsilon, \lambda_{\mathfrak{c}})$ (see Figure 2), where

- $V_{\mathfrak{c}} = \{t_u \mid u \in \{0,1\}^*\} \cup \{s^i_A \mid i \in \mathbb{N} \wedge A \subseteq \mathbb{N}\}$,
- $E_{\mathfrak{c}} = \{(t_u, t_{u0}), (t_u, t_{u1}) \mid u \in \{0,1\}^*\} \cup \{(t_\varepsilon, s^0_A) \mid A \subseteq \mathbb{N}\} \cup \{(s^i_A, s^{i+1}_A) \mid A \subseteq \mathbb{N}, i \in \mathbb{N}\}$,

[6]The depth is the maximal nesting of quantifiers, Boolean connectives, and temporal operators.

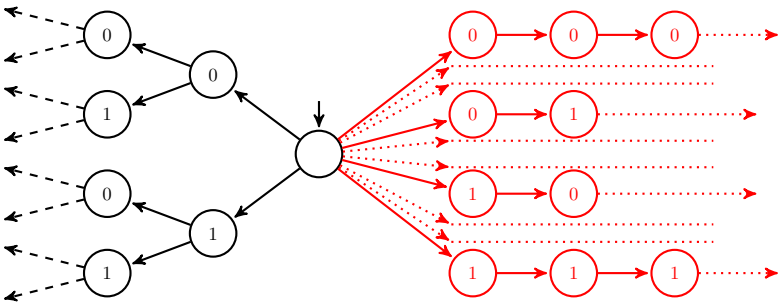


FIGURE 2. A depiction of $\mathcal{T}_{\mathfrak{c}}$. Vertices in black (on the left including the initial vertex) are labelled by `fbt`, those in red (on the right, excluding the initial vertex) are labelled by `set`.

- and the labelling $\lambda_{\mathfrak{c}}$ is defined as
  - $\lambda_{\mathfrak{c}}(t_\varepsilon) = \{\texttt{fbt}\}$
  - $\lambda_{\mathfrak{c}}(t_{u\cdot 0}) = \{\texttt{fbt}, 0\}$
  - $\lambda_{\mathfrak{c}}(t_{u\cdot 1}) = \{\texttt{fbt}, 1\}$, and
  - $\lambda_{\mathfrak{c}}(s^i_A) = \begin{cases} \{\texttt{set}, 0\} & \text{if } i \notin A, \\ \{\texttt{set}, 1\} & \text{if } i \in A. \end{cases}$

Lemma 5.4. *There is a satisfiable HyperCTL* sentence $\varphi_{\mathfrak{c}}$ that has only models of cardinality at least $\mathfrak{c}$.*

Proof. The formula $\varphi_{\mathfrak{c}}$ is defined as the conjunction of the formulas below:

(1) The label of the initial vertex is $\{\texttt{fbt}\}$ and the labels of non-initial vertices are $\{\texttt{fbt}, 0\}$, $\{\texttt{fbt}, 1\}$, $\{\texttt{set}, 0\}$, or $\{\texttt{set}, 1\}$:

$$\forall \pi.\ (\texttt{fbt}_\pi \wedge \neg 0_\pi \wedge \neg 1_\pi \wedge \neg \texttt{set}_\pi) \wedge \mathbf{X}\,\mathbf{G}\left((\texttt{set}_\pi \leftrightarrow \neg \texttt{fbt}_\pi) \wedge (0_\pi \leftrightarrow \neg 1_\pi)\right)$$

(2) All `fbt`-labelled vertices have a successor with label $\{\texttt{fbt}, 0\}$ and one with label $\{\texttt{fbt}, 1\}$, and all `fbt`-labelled vertices that are additionally labelled by 0 or 1 have no `set`-labelled successor:

$$\forall \pi.\ \mathbf{G}\left(\texttt{fbt}_\pi \rightarrow ((\exists \pi_0.\ \mathbf{X}(\texttt{fbt}_{\pi_0} \wedge 0_{\pi_0})) \wedge (\exists \pi_1.\ \mathbf{X}(\texttt{fbt}_{\pi_1} \wedge 1_{\pi_1})) \wedge ((0_\pi \vee 1_\pi) \rightarrow \forall \pi'.\ \mathbf{X}\,\texttt{fbt}_{\pi'}))\right)$$

(3) From `set`-labeled vertices, only `set`-labeled vertices are reachable:

$$\forall \pi.\ \mathbf{G}(\texttt{set} \rightarrow \mathbf{G}\,\texttt{set})$$

(4) For every path of `fbt`-labelled vertices starting at a successor of the initial vertex, there is a path of `set`-labelled vertices (also starting at a successor of the initial vertex) with the same $\{0, 1\}$ labelling:

$$\forall \pi.\ \left((\mathbf{X}\,\texttt{fbt}_\pi) \rightarrow \exists \pi'.\ \mathbf{X}(\texttt{set}_{\pi'} \wedge \mathbf{G}(0_\pi \leftrightarrow 0_{\pi'}))\right)$$

(5) Any two paths starting in the same `set`-labelled vertex have the same sequence of labels:

$$\forall \pi.\ \mathbf{G}\left(\texttt{set}_\pi \rightarrow \forall \pi'.\ \mathbf{G}(0_\pi \leftrightarrow 0_{\pi'})\right).$$

It is easy to check that $\mathcal{T}_{\mathfrak{c}} \models \varphi_{\mathfrak{c}}$. Note however that it is not the only model of $\varphi_{\mathfrak{c}}$: for instance, some paths may be duplicated, or merged after some steps if their label sequences share a common suffix. So, consider an arbitrary transition system $\mathcal{T} = (V, E, v_I, \lambda)$ such that $\mathcal{T} \models \varphi_{\mathfrak{c}}$. By condition 2, for every set $A \subseteq \mathbb{N}$, there is a path ρ_A starting at a successor of v_I such that $\lambda(\rho_A(i)) = \{\texttt{fbt}, 1\}$ if $i \in A$ and $\lambda(\rho_A(i)) = \{\texttt{fbt}, 0\}$ if $i \notin A$. Condition 3 implies that there is also a `set`-labelled path ρ'_A such that ρ'_A starts at a successor of v_I, and has the same $\{0, 1\}$ labelling as ρ_A. Finally, by condition 4, if $A \neq B$ then $\rho'_A(0) \neq \rho'_B(0)$.

So, the initial vertex has at least as many successors as there are subsets of $\mathbb{N}$, i.e., at least $\mathfrak{c}$ many. □

Before moving to the proof that HyperCTL* satisfiability is Σ^2_1-hard, we introduce one last auxiliary formula that will be used in the reduction, showing that addition and multiplication can be defined in HyperCTL*, and in fact even in HyperLTL, as follows: Let $\mathrm{AP} = \{\texttt{arg1}, \texttt{arg2}, \texttt{res}, \texttt{add}, \texttt{mult}\}$ and let $T_{(+,\cdot)}$ be the set of all traces $t \in (2^{\mathrm{AP}})^\omega$ such that

- there are unique $n_1, n_2, n_3 \in \mathbb{N}$ with $\texttt{arg1} \in t(n_1)$, $\texttt{arg2} \in t(n_2)$, and $\texttt{res} \in t(n_3)$, and
- either $\texttt{add} \in t(n)$ and $\texttt{mult} \notin t(n)$ for all n and $n_1 + n_2 = n_3$, or $\texttt{mult} \in t(n)$ and $\texttt{add} \notin t(n)$ for all n and $n_1 \cdot n_2 = n_3$.

Lemma 5.5. *There is a HyperLTL sentence $\varphi_{(+,\cdot)}$ which has $T_{(+,\cdot)}$ as unique model.*

Proof. Consider the conjunction of the following HyperLTL sentences:

(1) For every trace t there are unique $n_1, n_2, n_3 \in \mathbb{N}$ with $\texttt{arg1} \in t(n_1)$, $\texttt{arg2} \in t(n_2)$, and $\texttt{res} \in t(n_3)$:

$$\forall \pi. \bigwedge_{a \in \{\texttt{arg1},\texttt{arg2},\texttt{res}\}} (\neg a_\pi)\, \mathbf{U} (a_\pi \wedge \mathbf{X}\, \mathbf{G}\, \neg a_\pi)$$

(2) Every trace t satisfies either $\texttt{add} \in t(n)$ and $\texttt{mult} \notin t(n)$ for all n or $\texttt{mult} \in t(n)$ and $\texttt{add} \notin t(n)$ for all n:

$$\forall \pi.\ \mathbf{G}(\texttt{add}_\pi \wedge \neg \texttt{mult}_\pi) \vee \mathbf{G}(\texttt{mult}_\pi \wedge \neg \texttt{add}_\pi)$$

In the following, we only consider traces satisfying these formulas, as all others are not part of a model. Thus, we will speak of addition traces (if add holds) and multiplication traces (if mult holds). Furthermore, every trace encodes two unique arguments (given by the positions n_1 and n_2 such that $\texttt{arg1} \in t(n_1)$ and $\texttt{arg2} \in t(n_2)$) and a unique result (the position n_3 such that $\texttt{res} \in t(n_3)$).

Next, we need to express that all possible arguments are represented in a model, i.e. for every n_1 and every n_2 there are two traces t with $\texttt{arg1} \in t(n_1)$ and $\texttt{arg2} \in t(n_2)$, one addition trace and one multiplication trace. We do so inductively.

(3) There are two traces with both arguments being zero (i.e. arg1 and arg2 hold in the first position), one for addition and one for multiplication:

$$\bigwedge_{a \in \{\texttt{add},\texttt{mult}\}} \exists \pi.\ a_\pi \wedge \texttt{arg1}_\pi \wedge \texttt{arg2}_\pi$$

(4) Now, we express that for every trace, say encoding the arguments n_1 and n_2, the argument combinations (n_1+1, n_2) and (n_1, n_2+1) are also represented in the model, again both for addition and multiplication (here we rely on the fact that either add or mult holds at every position, as specified above):

$$\forall \pi.\ \exists \pi_1, \pi_2.\ \left(\bigwedge_{i \in \{1,2\}} \texttt{add}_\pi \leftrightarrow \texttt{add}_{\pi_i} \right) \wedge \mathbf{F}(\texttt{arg1}_\pi \wedge \mathbf{X}\, \texttt{arg1}_{\pi_1}) \wedge \mathbf{F}(\texttt{arg2}_\pi \wedge \texttt{arg2}_{\pi_1}) \wedge \mathbf{F}(\texttt{arg1}_\pi \wedge \texttt{arg1}_{\pi_2}) \wedge \mathbf{F}(\texttt{arg2}_\pi \wedge \mathbf{X}\, \texttt{arg2}_{\pi_2})$$

Every model of these formulas contains a trace representing each possible combination of arguments, both for addition and multiplication.

To conclude, we need to express that the result in each trace is correct. We do so by capturing the inductive definition of addition in terms of repeated increments

(which can be expressed by the next operator) and the inductive definition of multiplication in terms of repeated addition. Formally, this is captured by the next formulas:

(5) For every trace t: if $\{\texttt{add}, \texttt{arg1}\} \subseteq t(0)$ then $\texttt{arg2}$ and $\texttt{res}$ have to hold at the same position (this captures $0 + n = n$):

$$\forall \pi.\ (\texttt{add}_\pi \wedge \texttt{arg1}_\pi) \rightarrow \mathbf{F}(\texttt{arg2}_\pi \wedge \texttt{res}_\pi)$$

(6) For each trace t with $\texttt{add} \in t(0)$, $\texttt{arg1} \in t(n_1)$, $\texttt{arg2} \in t(n_2)$, and $\texttt{res} \in t(n_3)$ such that $n_1 > 0$ there is a trace t' such that $\texttt{add} \in t'(0)$, $\texttt{arg1} \in t'(n_1 - 1)$, $\texttt{arg2} \in t'(n_2)$, and $\texttt{res} \in t'(n_3 - 1)$ (this captures $n_1 + n_2 = n_3 \Leftrightarrow n_1 - 1 + n_2 = n_3 - 1$ for $n_1 > 0$):

$$\begin{aligned}&\forall \pi.\ \exists \pi'.\ (\texttt{add}_\pi \wedge \neg \texttt{arg1}_\pi) \rightarrow \\ &\quad (\texttt{add}_{\pi'} \wedge \mathbf{F}(\mathbf{X}\, \texttt{arg1}_\pi \wedge \texttt{arg1}_{\pi'}) \wedge \mathbf{F}(\texttt{arg2}_\pi \wedge \texttt{arg2}_{\pi'}) \wedge \mathbf{F}(\mathbf{X}\, \texttt{res}_\pi \wedge \texttt{res}_{\pi'}))\end{aligned}$$

(7) For every trace t: if $\{\texttt{mult}, \texttt{arg1}\} \subseteq t(0)$ then also $\texttt{res} \in t(0)$ (this captures $0 \cdot n = 0$):

$$\forall \pi.\ (\texttt{mult}_\pi \wedge \texttt{arg1}_\pi) \rightarrow \texttt{res}_\pi$$

(8) Similarly, for each trace t with $\texttt{mult} \in t(0)$, $\texttt{arg1} \in t(n_1)$, $\texttt{arg2} \in t(n_2)$, and $\texttt{res} \in t(n_3)$ such that $n_1 > 0$ there is a trace t' such that $\texttt{mult} \in t'(0)$, $\texttt{arg1} \in t'(n_1 - 1)$, $\texttt{arg2} \in t'(n_2)$, and $\texttt{res} \in t'(n_3 - n_2)$. The latter requirement is expressed by the existence of a trace t'' with $\texttt{add} \in t''(0)$, $\texttt{arg2} \in t''(n_2)$, $\texttt{res} \in t''(n_3)$, and $\texttt{arg1}$ holding in t'' at the same time as $\texttt{res}$ in t', which implies $\texttt{res} \in t'(n_3 - n_2)$. Altogether, this captures $n_1 \cdot n_2 = n_3 \Leftrightarrow (n_1 - 1) \cdot n_2 = n_3 - n_2$ for $n_1 > 0$.

$$\begin{aligned}&\forall \pi.\ \exists \pi', \pi''.\ (\texttt{mult}_\pi \wedge \neg \texttt{arg1}_\pi) \rightarrow (\texttt{mult}_{\pi'} \wedge \texttt{add}_{\pi''} \wedge \\ &\qquad \mathbf{F}(\mathbf{X}\, \texttt{arg1}_\pi \wedge \texttt{arg1}_{\pi'}) \wedge \mathbf{F}(\texttt{arg2}_\pi \wedge \texttt{arg2}_{\pi'} \wedge \texttt{arg2}_{\pi''}) \wedge \\ &\qquad \mathbf{F}(\texttt{res}_{\pi'} \wedge \texttt{arg1}_{\pi''}) \wedge \mathbf{F}(\texttt{res}_\pi \wedge \texttt{res}_{\pi''}))\end{aligned}$$

Now, $T_{(+,\cdot)}$ is a model of the conjunction $\varphi_{(+,\cdot)}$ of these eight formulas. Conversely, every model of $\varphi_{(+,\cdot)}$ contains all possible combinations of arguments (both for addition and multiplication) due to Formulas (3) and (4). Now, Formulas (5) to (8) ensure that the result is *correct* on these traces. Altogether, this implies that $T_{(+,\cdot)}$ is the unique model of $\varphi_{(+,\cdot)}$. □

To establish Σ^2_1-hardness, we give an encoding of formulas of existential third-order arithmetic into HyperCTL*, i.e. every formula of the form $\exists x_1.\ \ldots \exists x_n.\ \psi$ where $x_1, \ldots, x_n$ are third-order variables and ψ is a sentence of second-order arithmetic can be translated into a HyperCTL* sentence.

As explained in section 2, we can (and do for the remainder of the section) assume that first-order (type 0) variables range over natural numbers, second-order (type 1) variables range over sets of natural numbers, and third-order (type 2) variables range over sets of sets of natural numbers.

Lemma 5.6. *One can effectively translate sentences φ of existential third-order arithmetic into HyperCTL* sentences φ' such that $(\mathbb{N}, +, \cdot, <, \in)$ is a model of φ if and only if φ' is satisfiable.*

Proof. The idea of the proof is as follows. We represent sets of natural numbers as infinite paths with labels in $\{0, 1\}$, so that quantification over sets of natural numbers in ψ can be replaced by HyperCTL* path quantification. First-order quantification is handled in the same way, but using paths where exactly one vertex is labelled 1. In particular we encode first- and second-order variables x of φ as path variables π_x of φ'. For this to work, we need to make sure that every possible set has a path representative in the transition system (possibly several isomorphic

ones). This is where formula $\varphi_{\mathfrak{c}}$ defined in 5.4 is used. For arithmetical operations, we rely on the formula $\varphi_{(+,\cdot)}$ from 5.5. Finally, we associate with every existentially quantified third-order variable x_i an atomic proposition a_i, so that for a second-order variable y, the atomic formula $y \in x_i$ is interpreted as the atomic proposition a_i being true on the second vertex of π_y. This is all explained in more details below.

Let $\varphi = \exists x_1. \ \ldots \exists x_n. \ \psi$ where $x_1, \ldots, x_n$ are third-order variables and ψ is a formula of second-order arithmetic. We use the atomic propositions

$$\mathrm{AP} = \{a_1, \ldots, a_n, 0, 1, \mathtt{set}, \mathtt{fbt}, \mathtt{arg1}, \mathtt{arg2}, \mathtt{res}, \mathtt{mult}, \mathtt{add}\}.$$

Given an interpretation $\nu : \{x_1, \ldots, x_n\} \to 2^{(2^{\mathbb{N}})}$ of the third-order variables of φ, we denote by $\mathcal{T}_\nu$ the transition system over AP obtained as follows: We start from $\mathcal{T}_{\mathfrak{c}}$, and extend it with an $\{a_1, \ldots, a_n\}$-labelling by setting $a_i \in \lambda(\rho_A(0))$ if $A \in \nu(x_i)$; then, we add to this transition system all traces in $T_{(+,\cdot)}$ as disjoint paths below the initial vertex.

From the formulas $\varphi_{\mathfrak{c}}$ and $\varphi_{(+,\cdot)}$ defined in Lemmas 5.4 and 5.5, it is not difficult to construct a formula $\varphi_{(\mathfrak{c},+,\cdot)}$ such that:

- For all $\nu : \{x_1, \ldots, x_n\} \to 2^{(2^{\mathbb{N}})}$, the transition system $\mathcal{T}_\nu$ is a model of $\varphi_{(\mathfrak{c},+,\cdot)}$.
- Conversely, in any model $\mathcal{T} = (V, E, v_I, \lambda)$ of $\varphi_{(\mathfrak{c},+,\cdot)}$, the following conditions are satisfied:
  (1) For every path ρ starting at a $\mathtt{set}$-labelled successor of the initial vertex v_I, the vertex $\rho(0)$ has a label of the form $\lambda(\rho(0)) = \{\mathtt{set}, b\} \cup \ell$ with $b \in \{0, 1\}$ and $\ell \subseteq \{a_1, \ldots, a_n\}$, and every vertex $\rho(i)$ with $i > 0$ has a label $\lambda(\rho(i)) = \{\mathtt{set}, 0\}$ or $\lambda(\rho(i)) = \{\mathtt{set}, 1\}$.
  (2) For every $A \subseteq \mathbb{N}$, there exists a $\mathtt{set}$-labelled path ρ_A starting at a successor of v_I such that $1 \in \lambda(\rho_A(i))$ if $i \in A$, and $0 \in \lambda(\rho_A(i))$ if $i \notin A$. Moreover, all such paths have the same $\{a_1, \ldots, a_n\}$ labelling; this can be expressed by the formula

$$\forall \pi, \pi'. \ \mathbf{X}\left(\left(\mathbf{G}(\mathtt{set}_\pi \wedge \mathtt{set}_{\pi'} \wedge (1_\pi \leftrightarrow 1_{\pi'}))\right) \to \bigwedge\nolimits_{a \in \{a_1, \ldots, a_n\}} a_\pi \leftrightarrow a_{\pi'}\right).$$

  (3) For every path ρ starting at an $\mathtt{add}$- or $\mathtt{mult}$-labelled successor of the initial vertex, the label sequence $\lambda(\rho(0))\lambda(\rho(1))\cdots$ of ρ is in $T_{(+,\cdot)}$.
  (4) Conversely, for every trace $t \in T_{(+,\cdot)}$, there exists a path ρ starting at a successor of the initial vertex such that $\lambda(\rho(0))\lambda(\rho(1))\cdots = t$.

We then let $\varphi' = \varphi_{(\mathfrak{c},+,\cdot)} \wedge h(\psi)$, where $h(\psi)$ is defined inductively from the second-order body ψ of φ as follows:

- $h(\psi_1 \vee \psi_2) = h(\psi_1) \vee h(\psi_2)$.
- $h(\neg\psi_1) = \neg h(\psi_1)$.
- If x ranges over sets of natural numbers,

$$h(\exists x. \ \psi_1) = \exists \pi_x. \ ((\mathbf{X}\, \mathtt{set}_{\pi_x}) \wedge h(\psi_1)),$$

  and

$$h(\forall x. \ \psi_1) = \forall \pi_x. \ ((\mathbf{X}\, \mathtt{set}_{\pi_x}) \to h(\psi_1)).$$

- If x ranges over natural numbers,

$$h(\exists x. \ \psi_1) = \exists \pi_x. \ ((\mathbf{X}\, \mathtt{set}_{\pi_x}) \wedge \mathbf{X}(0_{\pi_x}\, \mathbf{U}(1_{\pi_x} \wedge \mathbf{X}\, \mathbf{G}\, 0_{\pi_x})) \wedge h(\psi_1)),$$

  and

$$h(\forall x. \ \psi_1) = \forall \pi_x. \ ((\mathbf{X}\, \mathtt{set}_{\pi_x}) \wedge \mathbf{X}(0_{\pi_x}\, \mathbf{U}(1_{\pi_x} \wedge \mathbf{X}\, \mathbf{G}\, 0_{\pi_x})) \to h(\psi_1)).$$

  Here, the subformula $0_{\pi_x}\, \mathbf{U}(1_{\pi_x} \wedge \mathbf{X}\, \mathbf{G}\, 0_{\pi_x})$ expresses that there is a single 1 on the trace assigned to π_x, i.e. the path represents a singleton set.

- If y ranges over sets of natural numbers, $h(y \in x_i) = \mathbf{X}(a_i)_{\pi_y}$.
- If x ranges over natural numbers and y over sets of natural numbers, $h(x \in y) = \mathbf{F}(1_{\pi_x} \wedge 1_{\pi_y})$.
- $h(x < y) = \mathbf{F}(1_{\pi_x} \wedge \mathbf{X}\,\mathbf{F}\,1_{\pi_y})$.
- $h(x + y = z) = \exists \pi.\ (\mathbf{X}\,\mathtt{add}_\pi) \wedge \mathbf{F}(\mathtt{arg1}_\pi \wedge 1_{\pi_x}) \wedge \mathbf{F}(\mathtt{arg2}_\pi \wedge 1_{\pi_y}) \wedge \mathbf{F}(\mathtt{res}_\pi \wedge 1_{\pi_z})$.
- $h(x \cdot y = z) = \exists \pi.\ (\mathbf{X}\,\mathtt{mult}_\pi) \wedge \mathbf{F}(\mathtt{arg1}_\pi \wedge 1_{\pi_x}) \wedge \mathbf{F}(\mathtt{arg2}_\pi \wedge 1_{\pi_y}) \wedge \mathbf{F}(\mathtt{res}_\pi \wedge 1_{\pi_z})$.

If ψ is true under some interpretation ν of $x_1, \dots, x_n$ as sets of sets of natural numbers, then the transition system $\mathcal{T}_\nu$ defined above is a model of φ'. Conversely, if $\mathcal{T} \models \varphi'$ for some transition system $\mathcal{T}$, then for all sets $A \subseteq \mathbb{N}$ there is a path ρ_A matching A in $\mathcal{T}$, and all such paths have the same $\{a_1, \dots, a_n\}$-labelling, so we can define an interpretation ν of $x_1, \dots, x_n$ by taking $A \in \nu(x_i)$ if and only if $a_i \in \lambda(\rho_A(0))$. Under this interpretation ψ holds, and thus φ is true, as first- and second-order quantification in $(\mathbb{N}, +, \cdot, <, \in)$ is mimicked by path quantification in $\mathcal{T}$. □

Now, we have all the tools at hand to prove the lower bound on the HyperCTL* satisfiability problem.

Lemma 5.7. *HyperCTL* satisfiability is Σ^2_1-hard.*

Proof. Let N be a Σ^2_1 set, i.e. $N = \{x \in \mathbb{N} \mid \exists x_0.\ \cdots \exists x_k.\ \psi(x, x_0, \dots, x_k)\}$ for some second-order arithmetic formula ψ with existentially quantified third-order variables x_i. For every $n \in \mathbb{N}$, we define the sentence

$$\varphi_n = \exists x_0.\ \cdots \exists x_k.\ \psi(n, x_0, \dots, x_k)\,.$$

Recall that every fixed natural number n is definable in first-order arithmetic, which is the reason we can use n in ψ.

Then φ_n is true if and only if $n \in N$. Combining this with 5.6, we obtain a computable function that maps any $n \in \mathbb{N}$ to a HyperCTL* formula φ'_n such that $n \in N$ if and only if φ'_n is satisfiable. □

5.4. **Variations of HyperCTL* Satisfiability.** The general HyperCTL* satisfiability problem, as studied above, asks for the existence of a model of arbitrary size. In the Σ^2_1-hardness proof we relied on uncountable models with infinite branching. Hence, it is natural to ask whether satisfiability is easier when we consider restricted classes of transition systems. In the remainder of this section, we study the following variations of satisfiability.

- The HyperCTL* *finitely-branching satisfiability problem*: given a HyperCTL* sentence, determine whether it has a finitely-branching model.[7]
- The HyperCTL* *countable satisfiability problem*: given a HyperCTL* sentence, determine whether it has a countable model.

But let us again begin with finite-state satisfiability, i.e., the question whether a given HyperCTL* sentence is satisfied by a finite transition system. As for HyperLTL (see 3.5), finite-state satisfiability is much simpler, but still undecidable. In fact, the lower bound is directly inherited from HyperLTL while the argument for the upper bound is the same, as HyperCTL* model-checking is also decidable.

Proposition 5.8 ([FRS15, FH16]). HyperCTL* finite-state satisfiability is Σ^0_1-complete.

Next, we show that the complexity of HyperCTL* finitely-branching satisfiability and countable satisfiability lies between that of finite-state satisfiability and general satisfiability: both are equivalent to *truth in second-order arithmetic*, that is, the

[7] A transition system is finitely-branching, if every vertex has only finitely many successors.

problem of deciding whether a given sentence of second-order arithmetic is satisfied in the standard model $(\mathbb{N}, 0, 1, +, \cdot, <, \in)$ of second-order arithmetic.

Theorem 5.9. *All of the following problems are effectively interreducible:*

(1) *HyperCTL* countable satisfiability.*
(2) *HyperCTL* finitely-branching satisfiability.*
(3) *Truth in second-order arithmetic.*

To prove 5.9, we show the implication (1) $\Rightarrow$ (3) in 5.10 and the implication (2) $\Rightarrow$ (3) in 5.11. Then, in 5.15 we show both converse implications simultaneously.

We start by showing that countable satisfiability can be effectively reduced to truth in second-order arithmetic. As every countable set is in bijection with the natural numbers, countable satisfiability asks for the existence of a model whose set of vertices is the set of natural numbers. This can easily be expressed in second-order arithmetic, leading to a fairly straightforward reduction to truth in second-order arithmetic.

Lemma 5.10. *There is an effective reduction from HyperCTL* countable satisfiability to truth in second-order arithmetic.*

Proof. Let φ be a HyperCTL* sentence. We construct a sentence φ^c of second-order arithmetic such that $(\mathbb{N}, 0, 1, +, \cdot, <, \in) \models \varphi^c$ if and only if φ has a countable model, or, equivalently, if and only if φ has a model of the form $\mathcal{T} = (\mathbb{N}, E, 0, \lambda)$ with vertex set $\mathbb{N}$, which implies that the set E of edges is a subset of $\mathbb{N} \times \mathbb{N}$. Note that we assume (w.l.o.g.) that the initial vertex is 0. The labeling function λ maps each natural number (that is, each vertex) to a set of atomic propositions. We assume a fixed encoding of valuations in 2^{AP} as natural numbers in $\{0, \dots, |2^{\mathrm{AP}}| - 1\}$, so that we can equivalently view λ as a function $\lambda : \mathbb{N} \to \mathbb{N}$ such that $\lambda(n) < |2^{\mathrm{AP}}|$ for all $n \in \mathbb{N}$. Note that binary relations over $\mathbb{N}$ can be encoded by functions from natural numbers to natural numbers, and the encoding can be implemented in first-order arithmetic.

The formula φ^c is defined as

$$\varphi^c = \exists E.\, \exists \lambda.\, (\forall x.\, \exists y.\, (x, y) \in E) \wedge (\forall x.\, \lambda(x) < |2^{\mathrm{AP}}|) \wedge \varphi'(E, \lambda, 0)\,,$$

where E is a second-order variable ranging over subsets of $\mathbb{N} \times \mathbb{N}$, λ a second-order variable ranging over functions from $\mathbb{N} \to \mathbb{N}$, and $\varphi'(E, \lambda, i)$, defined below, expresses the fact that the transition system $(\mathbb{N}, E, 0, \lambda)$ is a model of φ.

We use the following abbreviations:

- Given a second-order variable f ranging over functions from $\mathbb{N}$ to $\mathbb{N}$, the formula $\mathit{path}(f, E) = \forall n.\, (f(n), f(n+1)) \in E$ expresses the fact that $f(0)f(1)f(2)\dots$ is a path in $(\mathbb{N}, E, 0, \lambda)$.
- Given second-order variables f and f' ranging over functions from $\mathbb{N}$ to $\mathbb{N}$ and a first-order variable i ranging over natural numbers, we let

  $$\mathit{branch}(f, f', i, E) = \mathit{path}(f, E) \wedge \mathit{path}(f', E) \wedge \forall j \leq i.\, f(j) = f'(j)\,.$$

  This formula is satisfied by paths f and f' if f and f' coincide up to (and including) position i. We will use to restrict path quantification to those that start at a given position of a given path.

We define φ' inductively from φ, therefore considering in general HyperCTL* formulas with free variables $\pi_1, \dots, \pi_k$, in which case the formula φ' has free variables $E, \lambda, f_{\pi_1}, \dots, f_{\pi_k}, i$. The variable i is interpreted as the current time point. If φ is a sentence, i is not free in φ', as we use 0 in that case. Also, the translation depends on an ordering of the free variables of φ, i.e. quantified paths start at position i of the largest variable, as path quantification depends on the context

of a formula with free variables. In the following, we indicate the ordering by the naming of the variables, i.e. we have $\pi_1 < \cdots < \pi_k$.

- $a'_{\pi_j}(E, \lambda, f_{\pi_1}, \ldots, f_{\pi_k}, i) = \bigvee_{\{v \in 2^{\mathrm{AP}} \mid a \in v\}} \lambda(f_{\pi_j}(i)) = [v]$, where $[v]$ is the encoding of v as a natural number.
- If $\varphi(\pi_1, \ldots, \pi_k) = \neg\psi$ then $\varphi'(E, \lambda, f_{\pi_1}, \ldots, f_{\pi_k}, i) = \neg(\psi'(E, \lambda, f_{\pi_1}, \ldots, f_{\pi_k}, i))$.
- If $\varphi(\pi_1, \ldots, \pi_k) = \psi_1 \vee \psi_2$ then

$$\varphi'(E, \lambda, f_{\pi_1}, \ldots, f_{\pi_k}, i) = (\psi'_1(E, \lambda, f_{\pi_1}, \ldots, f_{\pi_k}, i)) \vee (\psi'_2(E, \lambda, f_{\pi_1}, \ldots, f_{\pi_k}, i)).$$

- If $\varphi(\pi_1, \ldots, \pi_k) = \mathbf{X}\,\psi$, then we define

$$\varphi'(E, \lambda, f_{\pi_1}, \ldots, f_{\pi_k}, i) = \psi'(E, \lambda, f_{\pi_1}, \ldots, f_{\pi_k}, i+1)\,.$$

- If $\varphi(\pi_1, \ldots, \pi_k) = \psi_1\,\mathbf{U}\,\psi_2$, then we define

$$\varphi'(E, \lambda, _{\pi_1}, \ldots, f_{\pi_k}, i) = \exists j.\, j \geq i \wedge \psi'_2(E, \lambda, f_{\pi_1}, \ldots, f_{\pi_k}, j) \wedge \forall j'.\, (i \leq j' < j \rightarrow \psi'_1(E, \lambda, f_{\pi_1}, \ldots, f_{\pi_k}, j'))\,.$$

- If $\varphi = \exists \pi_1 . \psi(\pi_1)$ is a sentence, then we define

$$\varphi'(E, \lambda) = \exists f_{\pi_1}.\ \mathit{path}(f_{\pi_1}, E) \wedge f_{\pi_1}(0) = 0 \wedge \psi'(E, \lambda, f_{\pi_1}, 0)\,.$$

  Recall that f_{π_1} ranges over functions from $\mathbb{N}$ to $\mathbb{N}$ and note that the formula requires f to encode a path and to start at the initial vertex 0.

  If $\varphi(\pi_1, \ldots, \pi_k) = \exists \pi_{k+1}.\ \psi(\pi_1, \ldots, \pi_k, \pi_{k+1})$ with $k > 0$, then we define

$$\varphi'(E, \lambda, f_{\pi_1}, \ldots, f_{\pi_k}, i) = \exists f_{\pi_{k+1}}.\ \mathit{branch}(f_{\pi_{k+1}}, f_{\pi_k}, i, E) \wedge \psi'(E, \lambda, f_{\pi_1}, \ldots, f_{\pi_k}, f_{\pi_{k+1}}, i)\,.$$

  Here, we make use of the ordering of the free variables of φ, as the translated formula requires the function assigned to $f_{\pi_{k+1}}$ to encode a path branching of the path encoded by the function assigned to f_{π_k}.
- If $\varphi = \forall \pi_1.\ \psi(\pi_1)$ is a sentence, then we define

$$\varphi'(E, \lambda) = \forall f_{\pi_1}.\ (\mathit{path}(f_{\pi_1}, E) \wedge f_{\pi_1}(0) = 0) \rightarrow \psi'(E, \lambda, f_{\pi_1}, 0)\,.$$

  If $\varphi(\pi_1, \ldots, \pi_k) = \forall \pi_{k+1}.\ \psi(\pi_1, \ldots, \pi_k, \pi_{k+1})$ with $k > 0$, then we define

$$\varphi'(E, \lambda, f_{\pi_1}, \ldots, f_{\pi_k}, i) = \forall f_{\pi_{k+1}}.\ \mathit{branch}(f_{\pi_{k+1}}, f_{\pi_k}, i, E) \rightarrow \psi'(E, \lambda, f_{\pi_1}, \ldots, f_{\pi_k}, f_{\pi_{k+1}}, i)$$

Now, φ has a countable model if and only if the second-order sentence φ^c is true in $(\mathbb{N}, 0, 1, +, \cdot, <, \in)$. □

Since every finitely-branching model has countably many vertices that are reachable from the initial vertex, the previous proof can be easily adapted for the case of finitely-branching satisfiability.

Lemma 5.11. *There is an effective reduction from HyperCTL* finitely-branching satisfiability to truth in second-order arithmetic.*

Proof. Let φ be a HyperCTL* sentence. We construct a second-order arithmetic formula φ^{fb} such that $(\mathbb{N}, 0, 1, +, \cdot, <, \in) \models \varphi^{fb}$ if and only if φ has a finitely-branching model, which we can again assume without loss of generality to be of the form $\mathcal{T} = (\mathbb{N}, E, 0, \lambda)$, where the set of vertices is $\mathbb{N}$, the set E of edges is a subset of $\mathbb{N} \times \mathbb{N}$, the initial vertex is 0, and the labeling function λ is encoded as a function from $\mathbb{N}$ to $\mathbb{N}$.

The formula φ^{fb} is almost identical to φ^c, only adding the finite branching requirement:

$$\varphi^{fb} = \exists E.\ \exists \lambda.\ (\forall x.\ \exists y.\ (x, y) \in E) \wedge (\forall x.\ \exists y.\ \forall z.\ (x, z) \in E \rightarrow z < y) \wedge (\forall x.\ \lambda(x) < |2^{\mathrm{AP}}|) \wedge \varphi'(E, \lambda, 0)\,.$$

□

Now, we consider the converse, i.e. that truth in second-order arithmetic can be reduced to countable and finitely-branching satisfiability. To this end, we adapt the Σ^2_1-hardness proof for HyperCTL*. Recall that we constructed a formula whose models contain all $\{0,1\}$-labelled paths, which we used to encode the subsets of $\mathbb{N}$. In that proof, we needed to ensure that the initial vertices of all these paths are pairwise different in order to encode existential third-order quantification, which resulted in uncountably many successors of the initial vertex. Also, we used the traces in $T_{(+,\cdot)}$ to encode arithmetic operations.

Here, we only have to encode first- and second-order quantification, so we can drop the requirement on the initial vertices of the paths encoding subsets, which simplifies our construction and removes one source of infinite branching. However, there is a second source of infinite branching, i.e. the infinitely many traces in $T_{(+,\cdot)}$ which all start at successors of the initial vertex. This is unavoidable: To obtain formulas that always have finitely-branching models, we can no longer work with $T_{(+,\cdot)}$. We begin by explaining the reason for this and then explain how to adapt the construction to obtain the desired result.

Recall that we defined $T_{(+,\cdot)}$ over $\mathrm{AP} = \{\texttt{arg1}, \texttt{arg2}, \texttt{res}, \texttt{add}, \texttt{mult}\}$ as the set of all traces $t \in (2^{\mathrm{AP}})^\omega$ such that

- there are unique $n_1, n_2, n_3 \in \mathbb{N}$ with $\texttt{arg1} \in t(n_1)$, $\texttt{arg2} \in t(n_2)$, and $\texttt{res} \in t(n_3)$, and
- either $\texttt{add} \in t(n)$ and $\texttt{mult} \notin t(n)$ for all n and $n_1 + n_2 = n_3$, or $\texttt{mult} \in t(n)$ and $\texttt{add} \notin t(n)$ for all n and $n_1 \cdot n_2 = n_3$.

An application of Kőnig's Lemma [Kőn27] shows that there is no finitely-branching transition system whose set of traces is $T_{(+,\cdot)}$. The reason is that $T_{(+,\cdot)}$ is not (topologically) closed (see definitions below), while the set of traces of a finitely-branching transition system is always closed.

Let $\mathrm{Pref}(t) \subseteq (2^{\mathrm{AP}})^*$ denote the set of finite prefixes of a trace $t \in (2^{\mathrm{AP}})^\omega$. Furthermore, let $\mathrm{Pref}(T) = \bigcup_{t \in T} \mathrm{Pref}(t)$ be the set of finite prefixes of a set $T \subseteq (2^{\mathrm{AP}})^\omega$ of traces. The closure $\mathrm{cl}(T) \subseteq (2^{\mathrm{AP}})^\omega$ of such a set T is defined as

$$\mathrm{cl}(T) = \{t \in (2^{\mathrm{AP}})^\omega \mid \mathrm{Pref}(t) \subseteq \mathrm{Pref}(T)\}.$$

For example, $\{\texttt{add}\}^\omega \in \mathrm{cl}(T_{(+,\cdot)})$ and $\{\texttt{mult}\}^*\{\texttt{arg2}, \texttt{mult}\}\{\texttt{mult}\}^\omega \subseteq \mathrm{cl}(T_{(+,\cdot)})$. Note that we have $T \subseteq \mathrm{cl}(T)$ for every T. As usual, we say that T is closed if $T = \mathrm{cl}(T)$.

Let AP be finite and let $T \subseteq (2^{\mathrm{AP}})^\omega$ be closed. Furthermore, let $\mathcal{T}(T)$ be the finitely-branching transition system $(\mathrm{Pref}(T), E, \varepsilon, \lambda)$ with

$$E = \{(w, wv) \mid wv \in \mathrm{Pref}(T) \text{ and } v \in 2^{\mathrm{AP}}\},$$

$\lambda(\varepsilon) = \emptyset$, and $\lambda(wv) = v$ for all $wv \in \mathrm{Pref}(T)$ with $v \in 2^{\mathrm{AP}}$.

Remark 5.12. The set of traces of paths of $\mathcal{T}(T)$ starting at the successors of the initial vertex ε is exactly T.

In the following, we show that we can replace the use of $T_{(+,\cdot)}$ by $\mathrm{cl}(T_{(+,\cdot)})$ and still capture addition and multiplication in HyperLTL. We begin by characterising the difference between $T_{(+,\cdot)}$ and $\mathrm{cl}(T_{(+,\cdot)})$ and then show that $\mathrm{cl}(T_{(+,\cdot)})$ is also the unique model of some HyperLTL sentence $\varphi^{cl}_{(+,\cdot)}$.

Intuitively, a trace is in $\mathrm{cl}(T_{(+,\cdot)}) \setminus T_{(+,\cdot)}$ if at least one of the arguments (the propositions $\texttt{arg1}$ and $\texttt{arg2}$) are missing. In all but one case, this also implies that $\texttt{res}$ does not occur in the trace, as the position of $\texttt{res}$ is (almost) always greater than the positions of the arguments. The only exception is when $\texttt{mult}$ holds and $\texttt{res}$ holds at the first position, i.e. in the limit of traces encoding $0 \cdot n = n$ for n tending towards infinity.

Let D be the set of traces t over $\mathrm{AP} = \{\texttt{arg1}, \texttt{arg2}, \texttt{res}, \texttt{add}, \texttt{mult}\}$ such that

- for each $a \in \{\texttt{arg1}, \texttt{arg2}, \texttt{res}\}$ there is at most one n such that $a \in t(n)$, and
- either $\texttt{add} \in t(n)$ and $\texttt{mult} \notin t(n)$ for all n, or $\texttt{mult} \in t(n)$ and $\texttt{add} \notin t(n)$ for all n,
- there is at least one $a \in \{\texttt{arg1}, \texttt{arg2}\}$ such that $a \notin t(n)$ for all n.
- Furthermore, if there is an n such that $\texttt{res} \in t(n)$, then $\texttt{mult} \in t(0)$, $n = 0$, and either $\texttt{arg1} \in t(0)$ or $\texttt{arg2} \in t(0)$.

Remark 5.13. $\mathrm{cl}(T_{(+,\cdot)}) \setminus T_{(+,\cdot)} = D$.

Now, we show the analogue of 5.5 for $\mathrm{cl}(T_{(+,\cdot)})$.

Lemma 5.14. *There is a HyperLTL sentence $\varphi^{cl}_{(+,\cdot)}$ which has $\mathrm{cl}(T_{(+,\cdot)})$ as unique model.*

Proof. We adapt the formula $\varphi_{(+,\cdot)}$ presented in the proof of 5.5 having $T_{(+,\cdot)}$ as unique model. Consider the conjunction of the following HyperLTL sentences:

(1) For every trace t and every $a \in \{\texttt{arg1}, \texttt{arg2}, \texttt{res}\}$ there is at most one n such that $a \in t(n)$:

$$\forall \pi. \bigwedge_{a \in \{\texttt{arg1},\texttt{arg2},\texttt{res}\}} (\mathbf{G}\, \neg a_\pi) \vee (\neg a_\pi)\, \mathbf{U} (a_\pi \wedge \mathbf{X}\, \mathbf{G}\, \neg a_\pi)$$

(2) For all traces t: If both $\texttt{arg1}$ and $\texttt{arg2}$ appear in t, then also $\texttt{res}$ (this captures the fact that the position of $\texttt{res}$ is determined by the positions of $\texttt{arg1}$ and $\texttt{arg2}$):

$$\forall \pi.\ (\mathbf{F}\, \texttt{arg1}_\pi \wedge \mathbf{F}\, \texttt{arg2}_\pi) \rightarrow \mathbf{F}\, \texttt{res}_\pi$$

(3) Every trace t satisfies either $\texttt{add} \in t(n)$ and $\texttt{mult} \notin t(n)$ for all n or $\texttt{mult} \in t(n)$ and $\texttt{add} \notin t(n)$ for all n:

$$\forall \pi.\ \mathbf{G}(\texttt{add}_\pi \wedge \neg \texttt{mult}_\pi) \vee \mathbf{G}(\texttt{mult}_\pi \wedge \neg \texttt{add}_\pi)$$

(4) For all traces t: If there is an $a \in \{\texttt{arg1}, \texttt{arg2}\}$ such that $a \notin t(n)$ for all n, but $\texttt{res} \in t(n_3)$ for some n_3, then $\{\texttt{mult}, \texttt{res}\} \subseteq t(0)$ and $\{\texttt{arg1}, \texttt{arg2}\} \cap t(0) \neq \emptyset$:

$$\forall \pi. \left(\mathbf{F}\, \texttt{res}_\pi \wedge \bigvee_{a \in \{\texttt{arg1},\texttt{arg2}\}} \mathbf{G}\, \neg a_\pi \right) \rightarrow \left(\texttt{mult}_\pi \wedge \texttt{res}_\pi \wedge \bigvee_{a \in \{\texttt{arg1},\texttt{arg2}\}} a \right)$$

We again only consider traces satisfying these formulas in the remainder of the proof, as all others are not part of a model. Also, we again speak of addition traces (if $\texttt{add}$ holds) and multiplication traces (if $\texttt{mult}$ holds).

Furthermore, if a trace satisfies the (guard) formula $\varphi_g = \mathbf{F}\, \mathrm{arg}_1 \wedge \mathbf{F}\, \texttt{arg2}$, then it encodes two unique arguments (given by the unique positions n_1 and n_2 such that $\texttt{arg1} \in t(n_1)$ and $\texttt{arg2} \in t(n_2)$. As the above formulas are satisfied, such a trace also encodes a result via the unique position n_3 such that $\texttt{res} \in t(n_3)$.

As before, we next express that every combination of inputs is present:

(5) There are two traces with both arguments being zero, one for addition and one for multiplication:

$$\bigwedge_{a \in \{\texttt{add},\texttt{mult}\}} \exists \pi.\ a_\pi \wedge \texttt{arg1}_\pi \wedge \texttt{arg2}_\pi$$

(6) For every trace encoding the arguments n_1 and n_2, the argument combinations (n_1+1, n_2) and (n_1, n_2+1) are also represented in the model, again both for addition and multiplication (here we rely on the fact that either `add` or `mult` holds at every position, as specified above). Note however, that not every trace will encode two inputs, which is why we have to use the guard φ_g.

$$\forall \pi.\ \varphi_g \rightarrow \exists \pi_1, \pi_2.\ \left(\bigwedge_{i \in \{1,2\}} \mathtt{add}_{\pi} \leftrightarrow \mathtt{add}_{\pi_i} \right) \wedge \mathbf{F}(\mathtt{arg1}_{\pi} \wedge \mathbf{X}\, \mathtt{arg1}_{\pi_1}) \wedge \mathbf{F}(\mathtt{arg2}_{\pi} \wedge \mathtt{arg2}_{\pi_1}) \wedge \mathbf{F}(\mathtt{arg1}_{\pi} \wedge \mathtt{arg1}_{\pi_2}) \wedge \mathbf{F}(\mathtt{arg2}_{\pi} \wedge \mathbf{X}\, \mathtt{arg2}_{\pi_2})$$

Every model of these formulas contains a trace representing each possible combination of arguments, both for multiplication and addition.

To conclude, we need to express that the result in each trace is correct by again capturing the inductive definition of addition in terms of repeated increments and the inductive definition of multiplication in terms of repeated addition. The formulas differ from those in the proof of 5.5 only in the use of the guard φ_g.

(7) For every trace t: if $\{\mathtt{add}, \mathtt{arg1}\} \subseteq t(0)$ and `arg2` appears in t then `arg2` and `res` have to hold at the same position (this captures $0 + n = n$):

$$\forall \pi.\ (\varphi_g \wedge \mathtt{add} \wedge \mathtt{arg1}_{\pi}) \rightarrow \mathbf{F}(\mathtt{arg2}_{\pi} \wedge \mathtt{res}_{\pi})$$

(8) For each trace t with $\mathtt{add} \in t(0)$, $\mathtt{arg1} \in t(n_1)$, $\mathtt{arg2} \in t(n_2)$, and $\mathtt{res} \in t(n_3)$ such that $n_1 > 0$ there is a trace t' such that $\mathtt{add} \in t'(0)$, $\mathtt{arg1} \in t'(n_1 - 1)$, $\mathtt{arg2} \in t'(n_2)$, and $\mathtt{res} \in t'(n_3 - 1)$ (this captures $n_1 + n_2 = n_3 \Leftrightarrow n_1 - 1 + n_2 = n_3 - 1$ for $n_1 > 0$):

$$\forall \pi.\ \exists \pi'.\ (\varphi_g \wedge \mathtt{add}_{\pi} \wedge \neg \mathtt{arg1}_{\pi}) \rightarrow (\mathtt{add}_{\pi'} \wedge \mathbf{F}(\mathbf{X}\, \mathtt{arg1}_{\pi} \wedge \mathtt{arg1}_{\pi'}) \wedge \mathbf{F}(\mathtt{arg2}_{\pi} \wedge \mathtt{arg2}_{\pi'}) \wedge \mathbf{F}(\mathbf{X}\, \mathtt{res}_{\pi} \wedge \mathtt{res}_{\pi'}))$$

(9) For every trace t: if $\{\mathtt{mult}, \mathtt{arg1}\} \subseteq t(0)$ then also $\mathtt{res} \in t(0)$ (this captures $0 \cdot n = 0$):

$$\forall \pi.\ (\mathtt{mult} \wedge \mathtt{arg1}_{\pi}) \rightarrow \mathtt{res}_{\pi}$$

(10) Similarly, for each trace t with $\mathtt{mult} \in t(0)$, $\mathtt{arg1} \in t(n_1)$, $\mathtt{arg2} \in t(n_2)$, and $\mathtt{res} \in t(n_3)$ such that $n_1 > 0$ there is a trace t' such that $\mathtt{mult} \in t'(0)$, $\mathtt{arg1} \in t'(n_1 - 1)$, $\mathtt{arg2} \in t'(n_2)$, and $\mathtt{res} \in t'(n_3 - n_2)$. The latter requirement is expressed by the existence of a trace t'' with $\mathtt{add} \in t''(0)$, $\mathtt{arg2} \in t''(n_2)$, $\mathtt{res} \in t''(n_3)$, and `arg1` holding in t'' at the same time as `res` in t', which implies $\mathtt{res} \in t'(n_3 - n_2)$. Altogether, this captures $n_1 \cdot n_2 = n_3 \Leftrightarrow (n_1 - 1) \cdot n_2 = n_3 - n_2$ for $n_1 > 0$.

$$\forall \pi.\ \exists \pi', \pi''.\ (\varphi_g \wedge \mathtt{mult}_{\pi} \wedge \neg \mathtt{arg1}_{\pi}) \rightarrow (\mathtt{mult}_{\pi'} \wedge \mathtt{add}_{\pi''} \wedge \mathbf{F}(\mathbf{X}\, \mathtt{arg1}_{\pi} \wedge \mathtt{arg1}_{\pi'}) \wedge \mathbf{F}(\mathtt{arg2}_{\pi} \wedge \mathtt{arg2}_{\pi'} \wedge \mathtt{arg2}_{\pi''}) \wedge \mathbf{F}(\mathtt{res}_{\pi'} \wedge \mathtt{arg1}_{\pi''}) \wedge \mathbf{F}(\mathtt{res}_{\pi} \wedge \mathtt{res}_{\pi''}))$$

Now, $\mathrm{cl}(T_{(+,\cdot)})$ is a model of the conjunction $\varphi^{cl}_{(+,\cdot)}$ of these ten formulas. Conversely, every model of $\varphi^{cl}_{(+,\cdot)}$ contains all possible combinations of arguments (both for addition and multiplication) due to Formulas (5) and (6). Now, Formulas (7) to (10) ensure that the result is *correct* on these traces. Furthermore, all traces in D, but not more, are also contained due to the first four formulas. Altogether, this implies that $\mathrm{cl}(T_{(+,\cdot)})$ is the unique model of $\varphi^{cl}_{(+,\cdot)}$. □

We are now ready to prove the lower bounds for HyperCTL* countable and finitely-branching satisfiability.

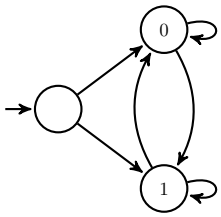


FIGURE 3. A depiction of $\mathcal{T}_f$. All vertices but the initial one are labelled by `fbt`.

Lemma 5.15. *There is an effective reduction from truth in second-order arithmetic to HyperCTL* countable and finitely-branching satisfiability.*

Proof. We proceed as in the proof of 5.6. Given a sentence φ of second-order arithmetic we construct a HyperCTL* formula φ' such that $(\mathbb{N}, +, \cdot, <, \in)$ is a model of φ if and only if φ' is satisfied by a countable and finitely-branching model.

As before, we represent sets of natural numbers as infinite paths with labels in $\{0, 1\}$, so quantification over sets of natural numbers and natural numbers is captured by path quantification. The major difference between our proof here and the one of 5.6 is that we do not need to deal with third-order quantification here. This means we only need to have every possible $\{0, 1\}$-labeled path in our models, but not with pairwise distinct initial vertices. In particular, the finite (and therefore finitely-branching) transition system $\mathcal{T}_f$ depicted in Figure 3 has all such paths.

For arithmetical operations, we rely on the HyperLTL sentence $\varphi^{cl}_{(+,\cdot)}$ from 5.14, with its unique model $\mathrm{cl}(T_{(+,\cdot)})$, and the transition system $\mathcal{T}(\mathrm{cl}(T_{(+,\cdot)}))$, which is countable, finitely-branching, and whose set of traces starting at the successors of the initial vertex is exactly $\mathrm{cl}(T_{(+,\cdot)})$. We combine $\mathcal{T}_f$ and $\mathcal{T}(\mathrm{cl}(T_{(+,\cdot)}))$ by identifying their respective initial vertices, but taking the disjoint union of all other vertices. The resulting transition system $\mathcal{T}_0$ contains all traces encoding the subsets of the natural numbers as well as the traces required to model arithmetical operations. Furthermore, it is still countable and finitely-branching.

Let $\mathrm{AP} = \{0, 1, \mathtt{fbt}, \mathtt{arg1}, \mathtt{arg2}, \mathtt{res}, \mathtt{mult}, \mathtt{add}\}$. Using parts of the formula $\varphi_{\mathfrak{c}}$ defined in 5.4 and the formula $\varphi^{cl}_{(+,\cdot)}$ defined in 5.14, it is not difficult to construct a formula $\varphi^{cl}_{(\mathfrak{c},+,\cdot)}$ such that:

- The transition system $\mathcal{T}_0$ is a model of $\varphi^{cl}_{(\mathfrak{c},+,\cdot)}$.
- Conversely, in any model $\mathcal{T} = (V, E, v_I, \lambda)$ of $\varphi^{cl}_{(\mathfrak{c},+,\cdot)}$, the following conditions are satisfied:
  (1) For every path ρ starting at a `fbt`-labelled successor of the initial vertex v_I, every vertex $\rho(i)$ with $i \geq 0$ has a label $\lambda(\rho(i)) = \{\mathtt{fbt}, 0\}$ or $\lambda(\rho(i)) = \{\mathtt{fbt}, 1\}$.
  (2) For every $A \subseteq \mathbb{N}$, there exists a `fbt`-labelled path ρ_A starting at a successor of v_I such that $1 \in \lambda(\rho_A(i))$ if $i \in A$, and $0 \in \lambda(\rho_A(i))$ if $i \notin A$.
  (3) For every path ρ starting at an `add`- or `mult`-labelled successor of the initial vertex, the label sequence $\lambda(\rho(0))\lambda(\rho(1))\cdots$ of ρ is in $\mathrm{cl}(T_{(+,\cdot)})$.
  (4) Conversely, for every trace $t \in \mathrm{cl}(T_{(+,\cdot)})$, there exists a path ρ starting at a successor of the initial vertex such that $\lambda(\rho(0))\lambda(\rho(1))\cdots = t$.

We then let $\varphi' = \varphi^{cl}_{(\mathfrak{c},+,\cdot)} \wedge h(\varphi)$, where $h(\varphi)$ is defined inductively from φ as in the proof of 5.6:

- $h(\psi_1 \vee \psi_2) = h(\psi_1) \vee h(\psi_2)$.

- $h(\neg\psi_1) = \neg h(\psi_1)$.
- If x ranges over sets of natural numbers,

  $$h(\exists x.\ \psi_1) = \exists \pi_x.\ ((\mathbf{X}\,\mathtt{fbt}_{\pi_x}) \wedge h(\psi_1)),$$

  and

  $$h(\forall x.\ \psi_1) = \forall \pi_x.\ ((\mathbf{X}\,\mathtt{fbt}_{\pi_x}) \rightarrow h(\psi_1)).$$
- If x ranges over natural numbers,

  $$h(\exists x.\ \psi_1) = \exists \pi_x.\ ((\mathbf{X}\,\mathtt{fbt}_{\pi_x}) \wedge \mathbf{X}(0_{\pi_x}\,\mathbf{U}(1_{\pi_x} \wedge \mathbf{X}\,\mathbf{G}\,0_{\pi_x})) \wedge h(\psi_1)),$$

  and

  $$h(\forall x.\ \psi_1) = \forall \pi_x.\ ((\mathbf{X}\,\mathtt{fbt}_{\pi_x}) \wedge \mathbf{X}(0_{\pi_x}\,\mathbf{U}(1_{\pi_x} \wedge \mathbf{X}\,\mathbf{G}\,0_{\pi_x})) \rightarrow h(\psi_1)).$$

  Here, the subformula $0_{\pi_x}\,\mathbf{U}(1_{\pi_x} \wedge \mathbf{X}\,\mathbf{G}\,0_{\pi_x})$ expresses that there is a single 1 on the trace assigned to π_x, i.e. the path represents a singleton set.
- If x ranges over natural numbers and y over sets of natural numbers, $h(x \in y) = \mathbf{F}(1_{\pi_x} \wedge 1_{\pi_y})$.
- $h(x < y) = \mathbf{F}(1_{\pi_x} \wedge \mathbf{X}\,\mathbf{F}\,1_{\pi_y})$.
- $h(x + y = z) = \exists \pi.\ (\mathbf{X}\,\mathtt{add}_\pi) \wedge \mathbf{F}(\mathtt{arg1}_\pi \wedge 1_{\pi_x}) \wedge \mathbf{F}(\mathtt{arg2}_\pi \wedge 1_{\pi_y}) \wedge \mathbf{F}(\mathtt{res}_\pi \wedge 1_{\pi_z})$.
- $h(x \cdot y = z) = \exists \pi.\ (\mathbf{X}\,\mathtt{mult}_\pi) \wedge \mathbf{F}(\mathtt{arg1}_\pi \wedge 1_{\pi_x}) \wedge \mathbf{F}(\mathtt{arg2}_\pi \wedge 1_{\pi_y}) \wedge \mathbf{F}(\mathtt{res}_\pi \wedge 1_{\pi_z})$.

If φ is true in $(\mathbb{N}, +, \cdot, <, \in)$, then the countable and finitely-branching transition system $\mathcal{T}_0$ defined above is a model of φ'. Conversely, if $\mathcal{T} \models \varphi'$ for some transition system $\mathcal{T}$, then for all sets $A \subseteq \mathbb{N}$ there is a path ρ_A matching A in $\mathcal{T}$ and trace quantification in $\mathcal{T}$ mimics first- and second-order in $(\mathbb{N}, +, \cdot, <, \in)$. Thus, φ is true in $(\mathbb{N}, +, \cdot, <, \in)$. □

Note that the preceding proof shows that even HyperCTL* bounded-branching satisfiability is equivalent to truth in second-order arithmetic, i.e., the question of whether a given sentence is satisfied by a transition system where each vertex has at most k successors, for some uniform $k \in \mathbb{N}$.

6. Related Work

The HyperLTL and HyperCTL* model checking problems have been shown decidable in the first paper introducing these logics [CFK$^+$14] and their exact complexity (and that of variants) has been determined in a series of further works [FRS15, Rab16, MZ20]. The HyperLTL satisfiability problem has been shown undecidable by Finkbeiner and Hahn [FHH18], but no upper bounds on the complexity were known. Similarly, Rabe has shown that HyperCTL* satisfiability is Σ^1_1-hard [Rab16], but again no upper bounds were known. Here, we settle the exact complexity of satisfiability for both logics as well as that of some variants.

Further complexity results have been obtained by Bonakdarpour and Finkbeiner for monitoring [BF18], by Finkbeiner et al. for synthesis [FHL$^+$20], and by Bonakdarpour and Sheinvald for standard automata-theoretic problems for hyperproperties represented by automata [BS23], while Winter and Zimmermann showed that the existence of computable Skolem functions for HyperLTL is decidable [WZ24]. Such functions yield explanations and counterexamples for HyperLTL model checking.

The techniques and results presented here have been generalized to second-order HyperLTL [BFFM23], i.e. HyperLTL with quantification over *sets* of traces. This logic allows to express important hyperproperties like common knowledge in multi-agent systems and asynchronous hyperproperties, which are not expressible in HyperLTL. Second-order quantification increases the already high expressiveness considerably: Satisfiability, finite-state satisfiability and model checking are

all equivalent to truth in third-order arithmetic [FZ25]. The intuitive reason is that second-order quantification over traces corresponds to quantification over sets of sets of natural numbers (as traces can encode characteristic sequences of such sets) and we have presented here an "implementation" of addition and multiplication in HyperLTL. These ingredients yield the lower bounds while an embedding of HyperLTL in third-order arithmetic yields the matching upper bounds.

Similarly, the techniques and results presented here have been generalized to HyperQPTL as well, which extends HyperLTL by quantification over propositions [Rab16]. With uniform quantification, HyperQPTL satisfiability is equivalent to truth in second-order arithmetic [RZ24] while finite-state satisfiability and model-checking have the same complexity as for HyperLTL [FH16, Rab16]. Non-uniform quantification makes HyperQPTL as expressive as second-order HyperLTL [RZ24], which implies that all three problems are equivalent to truth in third-order arithmetic.

The specification and verification of asynchronous hyperproperties, one of the motivations for studying second-order HyperLTL, have also been addressed by introducing dedicated (first-order) extensions of HyperLTL. In fact, there is a wide range of such logics [BHNdC23, BCB+21, BF23b, BBST24, BPS21, BPS22, GMO21, HBFS23, KSV23, KSV24, KMVZ18]). While some of these works contain partial complexity results, there is currently no full picture of the complexity of the standard verification problems for these logics.

Similarly, logics for probabilistic hyperproperties have been introduced [ÁB18, DWÁ+22, ÁBBD20, DFT20], again with partial complexity results.

Finally, there is another approach towards specifying hyperproperties, namely using team semantics for temporal logics [KMVZ18, KMVZ24]: Lück [Lüc20] studied the complexity of satisfiability and model checking for TeamLTL with Boolean negation and proved both problems to be equivalent to truth in third-order arithmetic. Kontinen and Sandström [KS21] generalized this result and showed that any logic between TeamLTL with Boolean negation and second-order logic inherits the same complexity results. However, the complexity of TeamLTL model checking without Boolean negation is a longstanding open problem [KMVZ18, KMVZ24].

7. Conclusion

In this work, we have settled the complexity of the satisfiability problems for HyperLTL and HyperCTL*. In both cases, we significantly increased the lower bounds, i.e. from Σ^0_1 and Σ^1_1 to Σ^1_1 and Σ^2_1, respectively, and presented the first upper bounds, which are tight in both cases. Along the way, we also determined the complexity of restricted variants, e.g. HyperLTL satisfiability restricted to ultimately periodic traces (or, equivalently, to finite traces) is still Σ^1_1-complete while HyperLTL and HyperCTL* satisfiability restricted to finite transition systems is Σ^0_1-complete. Furthermore, we proved that both countable and the finitely-branching satisfiability for HyperCTL* are as hard as truth in second-order arithmetic. As a key step in our proofs, we showed a tight bound of $\mathfrak{c}$ on the size of minimal models for satisfiable HyperCTL* sentences. Finally, we also showed that deciding membership in any level of the HyperLTL quantifier alternation hierarchy is Π^1_1-complete.

References

[AB16] Shreya Agrawal and Borzoo Bonakdarpour. Runtime verification of k-safety hyperproperties in HyperLTL. In *CSF 2016*, pages 239–252. IEEE Computer Society, 2016. doi:10.1109/CSF.2016.24.

[ÁB18] Erika Ábrahám and Borzoo Bonakdarpour. HyperPCTL: A temporal logic for probabilistic hyperproperties. In Annabelle McIver and András Horváth, editors,

QEST 2018, volume 11024 of *LNCS*, pages 20–35. Springer, 2018. doi:10.1007/978-3-319-99154-2_2.

[ÁBBD20] Erika Ábrahám, Ezio Bartocci, Borzoo Bonakdarpour, and Oyendrila Dobe. Probabilistic hyperproperties with nondeterminism. In Dang Van Hung and Oleg Sokolsky, editors, *ATVA 2020*, volume 12302 of *LNCS*, pages 518–534. Springer, 2020. doi:10.1007/978-3-030-59152-6_29.

[BBST24] Alberto Bombardelli, Laura Bozzelli, César Sánchez, and Stefano Tonetta. Unifying asynchronous logics for hyperproperties. In Siddharth Barman and Slawomir Lasota, editors, *FSTTCS 2024*, volume 323 of *LIPIcs*, pages 14:1–14:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024. doi:10.4230/LIPICS.FSTTCS.2024.14.

[BCB+21] Jan Baumeister, Norine Coenen, Borzoo Bonakdarpour, Bernd Finkbeiner, and César Sánchez. A temporal logic for asynchronous hyperproperties. In Alexandra Silva and K. Rustan M. Leino, editors, *CAV 2021, Part I*, volume 12759 of *LNCS*, pages 694–717. Springer, 2021. doi:10.1007/978-3-030-81685-8_33.

[BCF+22] Raven Beutner, David Carral, Bernd Finkbeiner, Jana Hofmann, and Markus Krötzsch. Deciding hyperproperties combined with functional specifications. In Christel Baier and Dana Fisman, editors, *LICS 2022*, pages 56:1–56:13. ACM, 2022. doi:10.1145/3531130.3533369.

[BDFH16] Gilles Barthe, Pedro R. D'Argenio, Bernd Finkbeiner, and Holger Hermanns. Facets of software doping. In Tiziana Margaria and Bernhard Steffen, editors, *ISoLA 2016, Part II*, volume 9953 of *LNCS*, pages 601–608, 2016. doi:10.1007/978-3-319-47169-3_46.

[BF16] Borzoo Bonakdarpour and Bernd Finkbeiner. Runtime verification for HyperLTL. In Yliès Falcone and César Sánchez, editors, *RV 2016*, volume 10012 of *LNCS*, pages 41–45. Springer, 2016. doi:10.1007/978-3-319-46982-9_4.

[BF18] Borzoo Bonakdarpour and Bernd Finkbeiner. The complexity of monitoring hyperproperties. In *CSF*, pages 162–174. IEEE Computer Society, 2018. doi:10.1109/CSF.2018.00019.

[BF20] Borzoo Bonakdarpour and Bernd Finkbeiner. Controller synthesis for hyperproperties. In *CSF 2020*, pages 366–379. IEEE, 2020. doi:10.1109/CSF49147.2020.00033.

[BF22] Raven Beutner and Bernd Finkbeiner. Software verification of hyperproperties beyond k-safety. In Sharon Shoham and Yakir Vizel, editors, *CAV 2022, Part I*, volume 13371 of *LNCS*, pages 341–362. Springer, 2022. doi:10.1007/978-3-031-13185-1_17.

[BF23a] Raven Beutner and Bernd Finkbeiner. Autohyper: Explicit-state model checking for HyperLTL. In Sriram Sankaranarayanan and Natasha Sharygina, editors, *TACAS 2023, Part I*, volume 13993 of *LNCS*, pages 145–163. Springer, 2023. doi:10.1007/978-3-031-30823-9_8.

[BF23b] Raven Beutner and Bernd Finkbeiner. HyperATL*: A logic for hyperproperties in multi-agent systems. *Log. Methods Comput. Sci.*, 19(2), 2023. doi:10.46298/LMCS-19(2:13)2023.

[BFFM23] Raven Beutner, Bernd Finkbeiner, Hadar Frenkel, and Niklas Metzger. Second-order hyperproperties. In Constantin Enea and Akash Lal, editors, *CAV 2023, Part II*, volume 13965 of *LNCS*, pages 309–332. Springer, 2023. doi:10.1007/978-3-031-37703-7_15.

[BHNdC23] Ezio Bartocci, Thomas A. Henzinger, Dejan Nickovic, and Ana Oliveira da Costa. Hypernode automata. In Guillermo A. Pérez and Jean-François Raskin, editors, *CONCUR 2023*, volume 279 of *LIPIcs*, pages 21:1–21:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023. doi:10.4230/LIPICS.CONCUR.2023.21.

[BPS21] Laura Bozzelli, Adriano Peron, and César Sánchez. Asynchronous extensions of HyperLTL. In *LICS 2021*, pages 1–13. IEEE, 2021. doi:10.1109/LICS52264.2021.9470583.

[BPS22] Laura Bozzelli, Adriano Peron, and César Sánchez. Expressiveness and decidability of temporal logics for asynchronous hyperproperties. In Bartek Klin, Slawomir Lasota, and Anca Muscholl, editors, *CONCUR 2022*, volume 243 of *LIPIcs*, pages 27:1–27:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022. doi:10.4230/LIPIcs.CONCUR.2022.27.

[BS23] Borzoo Bonakdarpour and Sarai Sheinvald. Finite-word hyperlanguages. *Inf. Comput.*, 295(Part B):104944, 2023. doi:10.1016/J.IC.2022.104944.

[BSB17] Noel Brett, Umair Siddique, and Borzoo Bonakdarpour. Rewriting-based runtime verification for alternation-free HyperLTL. In Axel Legay and Tiziana Margaria, editors, *TACAS 2017, Part II*, volume 10206 of *LNCS*, pages 77–93, 2017. doi:10.1007/978-3-662-54580-5_5.

[CB71] Rina S. Cohen and Janusz A. Brzozowski. Dot-depth of star-free events. *J. Comput. Syst. Sci.*, 5(1):1–16, 1971. doi:10.1016/S0022-0000(71)80003-X.

[CFH+21] Norine Coenen, Bernd Finkbeiner, Christopher Hahn, Jana Hofmann, and Yannick Schillo. Runtime enforcement of hyperproperties. In Zhe Hou and Vijay Ganesh, editors, *ATVA 2021*, volume 12971 of *LNCS*, pages 283–299. Springer, 2021. doi:10.1007/978-3-030-88885-5_19.

[CFK+14] Michael R. Clarkson, Bernd Finkbeiner, Masoud Koleini, Kristopher K. Micinski, Markus N. Rabe, and César Sánchez. Temporal logics for hyperproperties. In Martín Abadi and Steve Kremer, editors, *POST 2014*, volume 8414 of *LNCS*, pages 265–284. Springer, 2014. doi:10.1007/978-3-642-54792-8_15.

[CS10] Michael R. Clarkson and Fred B. Schneider. Hyperproperties. *J. Comput. Secur.*, 18(6):1157–1210, 2010. doi:10.3233/JCS-2009-0393.

[DFT20] Rayna Dimitrova, Bernd Finkbeiner, and Hazem Torfah. Probabilistic hyperproperties of Markov decision processes. In Dang Van Hung and Oleg Sokolsky, editors, *ATVA 2020*, volume 12302 of *LNCS*, pages 484–500. Springer, 2020. doi:10.1007/978-3-030-59152-6_27.

[DG08] Volker Diekert and Paul Gastin. First-order definable languages. In *Logic and Automata: History and Perspectives [in Honor of Wolfgang Thomas]*, volume 2 of *Texts in Logic and Games*, pages 261–306. Amsterdam University Press, 2008.

[DWÁ+22] Oyendrila Dobe, Lukas Wilke, Erika Ábrahám, Ezio Bartocci, and Borzoo Bonakdarpour. Probabilistic hyperproperties with rewards. In Jyotirmoy V. Deshmukh, Klaus Havelund, and Ivan Perez, editors, *NFM 2022*, volume 13260 of *LNCS*, pages 656–673. Springer, 2022. doi:10.1007/978-3-031-06773-0_35.

[EH86] E. Allen Emerson and Joseph Y. Halpern. "sometimes" and "not never" revisited: on branching versus linear time temporal logic. *J. ACM*, 33(1):151–178, 1986. doi:10.1145/4904.4999.

[FH16] Bernd Finkbeiner and Christopher Hahn. Deciding hyperproperties. In Josée Desharnais and Radha Jagadeesan, editors, *CONCUR 2016*, volume 59 of *LIPIcs*, pages 13:1–13:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016. doi:10.4230/LIPIcs.CONCUR.2016.13.

[FHH18] Bernd Finkbeiner, Christopher Hahn, and Tobias Hans. MGHyper: Checking satisfiability of HyperLTL formulas beyond the $\exists^*\forall^*$ fragment. In *ATVA 2018*, volume 11138 of *LNCS*, pages 521–527. Springer, 2018. doi:10.1007/978-3-030-01090-4_31.

[FHHT20] Bernd Finkbeiner, Christopher Hahn, Jana Hofmann, and Leander Tentrup. Realizing ω-regular hyperproperties. In Shuvendu K. Lahiri and Chao Wang, editors, *CAV 2020, Part II*, volume 12225 of *LNCS*, pages 40–63. Springer, 2020. doi:10.1007/978-3-030-53291-8_4.

[FHL+20] Bernd Finkbeiner, Christopher Hahn, Philip Lukert, Marvin Stenger, and Leander Tentrup. Synthesis from hyperproperties. *Acta Informatica*, 57(1-2):137–163, 2020. doi:10.1007/s00236-019-00358-2.

[FHS17] Bernd Finkbeiner, Christopher Hahn, and Marvin Stenger. EAHyper: Satisfiability, implication, and equivalence checking of hyperproperties. In Rupak Majumdar and Viktor Kuncak, editors, *CAV 2017, Part II*, volume 10427 of *LNCS*, pages 564–570. Springer, 2017. doi:10.1007/978-3-319-63390-9_29.

[FHST18] Bernd Finkbeiner, Christopher Hahn, Marvin Stenger, and Leander Tentrup. RVHyper: A runtime verification tool for temporal hyperproperties. In Dirk Beyer and Marieke Huisman, editors, *TACAS 2018, Part II*, volume 10806 of *LNCS*, pages 194–200. Springer, 2018. doi:10.1007/978-3-319-89963-3_11.

[Fin21] Bernd Finkbeiner. Model checking algorithms for hyperproperties (invited paper). In Fritz Henglein, Sharon Shoham, and Yakir Vizel, editors, *VMCAI 2021*, volume 12597 of *LNCS*, pages 3–16. Springer, 2021. doi:10.1007/978-3-030-67067-2_1.

[FKTZ21] Marie Fortin, Louwe B. Kuijer, Patrick Totzke, and Martin Zimmermann. HyperLTL satisfiability is Σ^1_1-complete, HyperCTL* satisfiability is Σ^2_1-complete. In Filippo Bonchi and Simon J. Puglisi, editors, *MFCS 2021*, volume 202 of *LIPIcs*, pages 47:1–47:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPIcs.MFCS.2021.47.

[FRS15] Bernd Finkbeiner, Markus N. Rabe, and César Sánchez. Algorithms for Model Checking HyperLTL and HyperCTL*. In Daniel Kroening and Corina S. Pasareanu, editors, *CAV 2015, Part I*, volume 9206 of *LNCS*, pages 30–48. Springer, 2015. doi:10.1007/978-3-319-21690-4_3.

[FZ17] Bernd Finkbeiner and Martin Zimmermann. The first-order logic of hyperproperties. In *STACS 2017*, volume 66 of *LIPIcs*, pages 30:1–30:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017. doi:10.4230/LIPIcs.STACS.2017.30.

[FZ25] Hadar Frenkel and Martin Zimmermann. The complexity of second-order HyperLTL. In Jörg Endrullis and Sylvain Schmitz, editors, *CSL 2025*, volume 326 of *LIPIcs*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025.

[GMO21] Jens Oliver Gutsfeld, Markus Müller-Olm, and Christoph Ohrem. Automata and fixpoints for asynchronous hyperproperties. *Proc. ACM Program. Lang.*, 5(POPL):1–29, 2021. doi:10.1145/3434319.

[Har85] David Harel. Recurring dominoes: Making the highly undecidable highly understandable. *North-Holland Mathematical Studies*, 102:51–71, 1985. doi:10.1016/S0304-0208(08)73075-5.

[HBFS23] Tzu-Han Hsu, Borzoo Bonakdarpour, Bernd Finkbeiner, and César Sánchez. Bounded model checking for asynchronous hyperproperties. In Sriram Sankaranarayanan and Natasha Sharygina, editors, *TACAS 2023, Part I*, volume 13993 of *LNCS*, pages 29–46. Springer, 2023. doi:10.1007/978-3-031-30823-9_2.

[Hin17] Peter G. Hinman. *Recursion-Theoretic Hierarchies*. Perspectives in Logic. Cambridge University Press, 2017. doi:10.1017/9781316717110.

[KMVZ18] Andreas Krebs, Arne Meier, Jonni Virtema, and Martin Zimmermann. Team semantics for the specification and verification of hyperproperties. In Igor Potapov, Paul G. Spirakis, and James Worrell, editors, *MFCS 2018*, volume 117 of *LIPIcs*, pages 10:1–10:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018. doi:10.4230/LIPIcs.MFCS.2018.10.

[KMVZ24] Andreas Krebs, Arne Meier, Jonni Virtema, and Martin Zimmermann. Synchronous team semantics for temporal logics. *arXiv*, 2409.18667, 2024. doi:10.48550/ARXIV.2409.18667.

[Kőn27] Dénes Kőnig. Über eine Schlussweise aus dem Endlichen ins Unendliche. *Acta litt. sci. Reg. Univ. Hung. Francisco-Josephinae, Sect. sci. math.*, 3(2-3):121–130, 1927.

[KS21] Juha Kontinen and Max Sandström. On the expressive power of TeamLTL and first-order team logic over hyperproperties. In Alexandra Silva, Renata Wassermann, and Ruy J. G. B. de Queiroz, editors, *WoLLIC 2021*, volume 13038 of *LNCS*, pages 302–318. Springer, 2021. doi:10.1007/978-3-030-88853-4_19.

[KSV23] Juha Kontinen, Max Sandström, and Jonni Virtema. Set semantics for asynchronous TeamLTL: Expressivity and complexity. In Jérôme Leroux, Sylvain Lombardy, and David Peleg, editors, *MFCS 2023*, volume 272 of *LIPIcs*, pages 60:1–60:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023. doi:10.4230/LIPICS.MFCS.2023.60.

[KSV24] Juha Kontinen, Max Sandström, and Jonni Virtema. A remark on the expressivity of asynchronous TeamLTL and HyperLTL. In Arne Meier and Magdalena Ortiz, editors, *FoIKS 2024*, volume 14589 of *LNCS*, pages 275–286. Springer, 2024. doi:10.1007/978-3-031-56940-1_15.

[Lüc20] Martin Lück. On the complexity of linear temporal logic with team semantics. *Theor. Comput. Sci.*, 837:1–25, 2020. doi:10.1016/j.tcs.2020.04.019.

[MZ20] Corto Mascle and Martin Zimmermann. The keys to decidable HyperLTL satisfiability: Small models or very simple formulas. In Maribel Fernández and Anca Muscholl, editors, *CSL 2020*, volume 152 of *LIPIcs*, pages 29:1–29:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020. doi:10.4230/LIPIcs.CSL.2020.29.

[Pnu77] Amir Pnueli. The temporal logic of programs. In *FOCS 1977*, pages 46–57. IEEE, 1977. doi:10.1109/SFCS.1977.32.

[Rab16] Markus N. Rabe. *A temporal logic approach to information-flow control*. PhD thesis, Saarland University, 2016. URL: http://scidok.sulb.uni-saarland.de/volltexte/2016/6387/.

[Rog87] Hartley Rogers. *Theory of Recursive Functions and Effective Computability*. MIT Press, Cambridge, MA, USA, 1987.

[RZ24] Gaëtan Regaud and Martin Zimmermann. The complexity of HyperQPTL. *arXiv*, 2024. doi:10.48550/ARXIV.2412.07341.

[Tho81] Wolfgang Thomas. A combinatorial approach to the theory of omega-automata. *Inf. Control.*, 48(3):261–283, 1981. doi:10.1016/S0019-9958(81)90663-X.

[Tho82] Wolfgang Thomas. Classifying regular events in symbolic logic. *J. Comput. Syst. Sci.*, 25(3):360–376, 1982. doi:10.1016/0022-0000(82)90016-2.

[WZ24] Sarah Winter and Martin Zimmermann. Tracy, traces, and transducers: Computable counterexamples and explanations for HyperLTL model-checking. *arXiv*, 2404.18280, 2024. doi:10.48550/ARXIV.2404.18280.